

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



ĐỒ ÁN MÔN HỌC KỸ THUẬT MÁY TÍNH
NGÀNH KỸ THUẬT MÁY TÍNH

**Hoạch định chuyển động cho rô-bốt sử
dụng tối ưu lỗi**

Học kỳ 251

Giảng viên hướng dẫn: PGS. TS Lê Hồng Trang
ThS. Trương Quỳnh Chi

STT	Họ và tên	MSSV	Ghi chú
1	Nguyễn Trần Đăng Khoa	2211635	
2	Hồ Đăng Khoa	2211588	

Thành phố Hồ Chí Minh, Tháng 12 năm 2025

Tóm tắt

Hoạch định chuyển động cho robot trong môi trường chứa vật cản vốn là một bài toán tối ưu hóa phi tuyến đầy thách thức do tính chất không lồi của không gian cấu hình. Trong khi các phương pháp dựa trên lấy mẫu truyền thống thường gặp hạn chế về tính tối ưu và khả năng xử lý các ràng buộc động lực học phức tạp, bài báo này đề xuất một phương pháp tiếp cận mới dựa trên Đồ thị các Tập Lồi (Graphs of Convex Sets - GCS). Bằng cách phân rã không gian tự do thành các vùng lồi hữu hạn và mô hình hóa quỹ đạo robot sử dụng đường cong Bézier, chúng tôi thiết lập bài toán dưới dạng tối ưu hóa hỗn hợp nguyên. Cuối cùng ta áp dụng kỹ thuật nới lỏng lồi (convex relaxation) chặt chẽ để chuyển đổi bài toán phức tạp này thành bài toán Lập trình nón bậc hai (Second-Order Cone Programming - SOCP) dễ giải. Kết quả thực nghiệm trên các hệ thống có số chiều cao chứng minh phương pháp này vượt trội hơn so với các thuật toán PRM truyền thống, giảm thiểu đáng kể thời gian tính toán trong khi vẫn đảm bảo tìm được quỹ đạo tối ưu toàn cục. Chúng tôi kiểm chứng GCS trên một loạt các môi trường với độ phức tạp tăng dần, từ các vật cản 2D đơn giản và mê cung phức tạp đến các thiết bị bay quadrotor động lực học và tay máy 7 bậc tự do (7-DoF) trong không gian nhiều chiều. Các thực nghiệm số chứng minh rằng GCS đạt hiệu năng vượt trội một cách nhất quán so với các bộ hoạch định dựa trên lấy mẫu được sử dụng rộng rãi (ví dụ: PRM). Cụ thể, trong các nhiệm vụ nhiều chiều, GCS giảm thời gian truy vấn trực tuyến lên đến một bậc so với PRM tiêu chuẩn, đồng thời tìm ra các quỹ đạo tối ưu toàn cục ngắn hơn đáng kể (giảm tới 40% độ dài) so với các bộ hoạch định dựa trên lấy mẫu đã được xử lý hậu kỳ [1].

Mục lục

1	Giới thiệu	1
1.1	Động lực và Thách thức Kỹ thuật	1
1.2	Goals	3
1.3	Scope	3
1.4	Cấu trúc luận văn	3
2	Kiến thức nền tảng	5
2.1	Giải tích Lồi và Tối ưu hóa	5
2.1.1	Tập Lồi (Convex Sets)	5
2.1.2	Nón Lồi và Đồng nhất hóa (Convex Cones and Homogenization)	6
2.1.2.1	Nón Phối cảnh và Đồng nhất hóa (Perspective Cones and Homogenization)	7
2.1.2.2	Nón Đối ngẫu (Dual Cones)	8
2.1.3	Hàm Lồi (Convex Functions)	9
2.1.4	Các Bài toán Tối ưu hóa (Optimization Problems)	9
2.1.4.1	Quy hoạch Lồi (Convex Optimization)	9
2.1.4.2	Quy hoạch Nón (Conic Optimization)	10
2.1.4.3	Quy hoạch Nguyên-Hỗn hợp (Mixed-Integer Programs)	10
2.2	Lý thuyết Đồ thị và Bài toán Đường đi Ngắn nhất	11
2.2.1	Cơ sở Lý thuyết Đồ thị	11
2.2.1.1	Các định nghĩa và khái niệm cơ bản	11

2.2.1.2	Tối ưu hóa tổ hợp trên đồ thị	12
2.2.2	Bài toán Đường đi Ngắn nhất (SPP)	13
2.2.2.1	Định nghĩa và các thuật toán cổ điển	13
2.2.2.2	Mô hình dòng mạng cho Bài toán Đường đi Ngắn nhất	14
2.3	Sinh quỹ đạo Động lực học (Kinodynamic)	15
2.3.1	Mô hình hóa Kinodynamic	15
2.3.2	Các ràng buộc về độ trơn và tính liên tục	16
2.3.3	Cơ sở lý thuyết về đường cong Bézier	17
2.4	Phân hoạch Lỗi	19
2.4.1	Định nghĩa	19
2.4.2	Phân loại Phương pháp Phân hoạch Lỗi	20
2.4.2.1	Phân hoạch Lỗi Chính xác (Exact Con- vex Decomposition – ECD)	20
2.4.2.2	Phân hoạch Lỗi Xấp xỉ (Approximate Con- vex Decomposition – ACD)	21
3	Related works	22
3.1	Sampling-Based Methods	22
3.2	Discussion	22
4	Phương pháp đề xuất	23
4.1	Phân hoạch không gian an toàn	23
4.2	Các Phương pháp Cốt lõi cho Phân hoạch Không gian Tự do	24
4.2.1	Phân hoạch Lỗi Xấp xỉ (ACD) của Đa giác	24
4.2.1.1	Các Khái niệm Cơ bản trong Phân rã Lỗi	24
4.2.1.2	Ý tưởng và Chi tiết Thuật toán	25
4.2.1.3	Các Thước đo Độ lõm	27

4.2.2	Iterative Regional Inflation by Semi-Definite Programming (IRIS)	28
4.2.2.1	Ý tưởng và Chi tiết Thuật toán	28
4.2.2.2	Đặc tính của Thuật toán	33
4.2.3	Visibility Clique Cover (VCC)	34
4.2.3.1	Ý tưởng và Chi tiết Thuật toán	34
4.3	Đồ thị của các Tập Lồi (Graphs of Convex Sets)	36
4.3.1	Công thức Đường đi Ngắn nhất trong Graph of Convex Sets	36
4.3.2	Phân tích Độ phức tạp	38
4.3.3	Mô hình chi tiết các đỉnh và cạnh	39
4.3.4	Đặc tả hàm chi phí	40
4.3.4.1	Độ dài Đường đi Hình học	40
4.3.4.2	Chi phí Quỹ đạo Động lực học	41
4.3.5	Xây dựng Bài toán Quy hoạch Lồi Nguyên Hỗn hợp thông qua Nói lỏng Phối cảnh	42
4.3.6	Lồi hóa Bài toán GCS thông qua Kỹ thuật RLT	45
4.3.6.1	Cơ sở Lý thuyết: Tính Đối ngẫu giữa Nón Phối cảnh và Bất đẳng thức Hợp lệ	45
4.3.6.2	Xây dựng các Ràng buộc Lồi	46
4.3.6.3	Công thức MICP Hoàn chỉnh	47
4.4	Thiết kế thực nghiệm	49
4.4.1	Môi trường 2D đơn giản	49
4.4.2	Môi trường mê cung	51
4.5	Kết quả và Phân tích	52
4.5.1	Phân tích Môi trường 2D đơn giản: Tác động của Phân hoạch lồi	52
4.5.1.1	Hiệu suất tính toán và Khả năng đáp ứng thời gian thực	56

4.5.1.2	Chất lượng Quỹ đạo và Sự đánh đổi giữa các Mục tiêu Tối ưu	57
4.5.2	Phân tích Thực nghiệm trên Môi trường Mê cung Phức tạp	58
4.5.2.1	Phân tích Hiệu suất Tính toán và Chi phí Tối ưu	59
4.5.2.2	Phân tích Đặc điểm Quỹ đạo và Tính Tối ưu Toàn cục	60
5	Conclusion	62
5.1	Summary	62
5.2	Future Work	62
	Tài liệu tham khảo	63
	Appendix A ...	66
A.1		66
A.2		66

Danh mục hình ảnh

Hình 1.1	Boston Dynamics' Atlas và Amazon's Robin	2
Hình 2.1	Ví dụ về Tập Lỗi	5
Hình 2.2	Ví dụ về Bao Lỗi	6
Hình 4.1	Ví dụ ACD	26
Hình 4.2	Mô tả quá trình đệ quy	26
Hình 4.3	Trường hợp không đảm bảo tính đơn điệu của ACD . .	27
Hình 4.4	Độ lõm Lai (Hybrid Concavity – H-Concavity)	28
Hình 4.5	Siêu phẳng Phân cách	29
Hình 4.6	Ellipsoid Nội tiếp Cực đại	31
Hình 4.7	Vòng lặp IRIS	32
Hình 4.8	Độ phức tạp thời gian của IRIS	33
Hình 4.9	VCC Overview	34
Hình 4.10	So sánh các phương pháp phân hoạch	53
Hình 4.11	So sánh các quỹ đạo chuyển động cho từng phương pháp phân hoạch	55
Hình 4.12	Quỹ đạo chuyển động cho 1 số mê cung	59

Danh mục bảng biểu

Bảng 4.1	So sánh hiệu năng giữa các phương pháp lập kế hoạch quỹ đạo	55
Bảng 4.2	Thống kê kết quả thực nghiệm trên 10 mẫu mê cung ngẫu nhiên	59

Chương 1

Giới thiệu

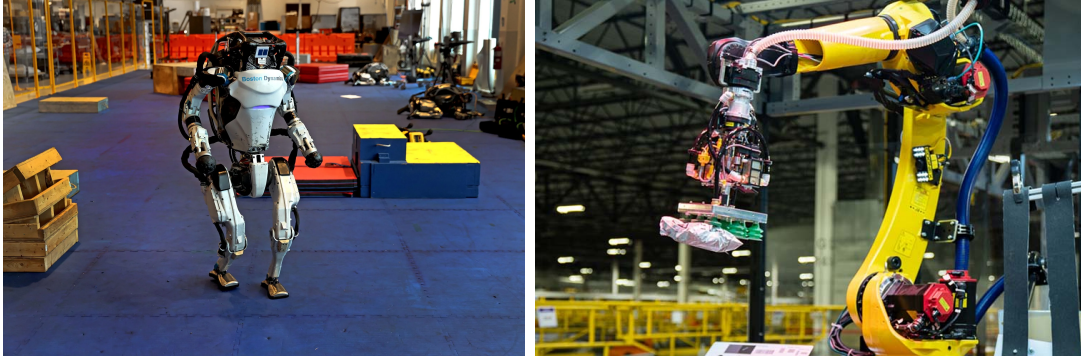
Trong chương 1, tổng quan, mục tiêu và các mục đích của đề tài nghiên cứu được trình bày. Bố cục của báo cáo cũng được giới thiệu.

1.1 Động lực và Thách thức Kỹ thuật

Việc đưa ra các quyết định tối ưu trong thời gian thực là một nền tảng cốt lõi của kỹ thuật hiện đại. Trong nhiều hệ thống phức tạp, đặc biệt là trong lĩnh vực robot tiên tiến, yêu cầu kết hợp chặt chẽ giữa **các quyết định rời rạc** và **điều khiển liên tục** trở nên hết sức rõ ràng.

- **Robot hình người (ví dụ: Atlas của Boston Dynamics):** Việc di chuyển trên địa hình phức tạp, chẳng hạn như các bài toán parkour, đòi hỏi robot phải đồng thời lựa chọn vị trí đặt chân (quyết định rời rạc) và tính toán quỹ đạo chuyển động cho trọng tâm cũng như các chi (điều khiển liên tục). Hiện nay, nhiều phương pháp vẫn phụ thuộc vào việc “lập trình thủ công” các quyết định rời rạc, điều này làm giảm đáng kể mức độ tự chủ của robot khi hoạt động trong những môi trường mới hoặc không xác định [2].
- **Robot công nghiệp (ví dụ: Robin của Amazon):** Hiệu quả của quá trình phân loại hàng hóa phụ thuộc vào việc tối ưu đồng thời thứ tự lựa chọn các thùng chứa (rời rạc) và quỹ đạo chuyển động của

cánh tay robot (liên tục) [3]. Chỉ cần cải thiện 1% tốc độ vận hành thông qua tối ưu hóa tốt hơn cũng có thể tương đương với việc xử lý thêm hàng triệu kiện hàng mỗi năm.



Hình 1.1: ***Bên trái:** Robot Atlas của Boston Dynamics thực hiện parkour, yêu cầu lập kế hoạch bước chân rời rạc và tối ưu quỹ đạo liên tục. **Bên phải:** Cánh tay robot Robin của Amazon phân loại hàng hóa, trong đó việc lựa chọn thùng chứa rời rạc và quỹ đạo chuyển động liên tục cần được tối ưu đồng thời.*

Tuy nhiên, việc giải quyết đồng thời các bài toán kết hợp này vẫn là một thách thức mở. Trong thực tế, các phương pháp hiện nay thường phải dựa vào sự can thiệp thủ công của con người hoặc chấp nhận các nghiệm dưới mức tối ưu.

Về mặt thuật toán, việc lựa chọn phương pháp lập kế hoạch chuyển động buộc các nhà nghiên cứu phải đánh đổi giữa nhiều yếu tố mâu thuẫn nhau, bao gồm số chiều của không gian trạng thái, các ràng buộc động học–động lực học và tính đầy đủ của thuật toán.

- **Tối ưu hóa quỹ đạo:** Các phương pháp này có ưu thế trong việc xử lý động lực học của robot và các bài toán có không gian trạng thái nhiều chiều. Tuy nhiên, khi môi trường chứa nhiều vật cản khiến bài toán trở nên phi lồi, chúng thường dễ rơi vào **cực tiểu cục bộ**, dẫn đến việc không tìm được quỹ đạo tránh va chạm [4].
- **Các phương pháp lập kế hoạch dựa trên lấy mẫu:** Để khắc phục hạn chế trên, cộng đồng robot học thường sử dụng các phương

pháp lấy mẫu như RRT hoặc PRM nhờ vào tính “đầy đủ theo xác suất” (probabilistic completeness). Tuy nhiên, cái giá phải trả là các quỹ đạo thu được thường **kém tối ưu**, và việc áp đặt các **ràng buộc vi phân liên tục** lên các mẫu rời rạc vẫn là một bài toán rất khó [5].

Lập trình lồi hỗn hợp số nguyên (Mixed-Integer Convex Programming – MICP) đã nổi lên như một hướng tiếp cận đầy hứa hẹn nhằm thu hẹp khoảng cách giữa hai nhóm phương pháp trên. MICP kết hợp được tính đầy đủ của các thuật toán dựa trên lấy mẫu với khả năng xử lý động lực học của tối ưu hóa quỹ đạo, đồng thời đảm bảo **tính tối ưu toàn cục** trong một khuôn khổ thống nhất. Tuy vậy, việc ứng dụng MICP trong thực tế vẫn bị hạn chế nghiêm trọng bởi chi phí tính toán rất cao, khi ngay cả những bài toán quy mô nhỏ cũng có thể mất tới vài phút để giải.

Mục tiêu của nghiên cứu này là phá vỡ rào cản tính toán đó. Chúng tôi tập trung vào một lớp bài toán lập kế hoạch chuyển động quan trọng có chứa các ràng buộc vi phân, và đề xuất một bộ lập kế hoạch dựa trên MICP có khả năng giải quyết các bài toán không gian nhiều chiều một cách đáng tin cậy chỉ trong vài giây, thông qua việc giải **một bài toán lồi duy nhất**.

1.2 Goals

1.3 Scope

1.4 Cấu trúc luận văn

Luận văn này được tổ chức thành năm chương, cụ thể như sau:

- **Chương 1** trình bày tổng quan về đề tài, bao gồm động lực nghiên cứu, mục tiêu cũng như phạm vi của luận văn.

- **Chương 2** tập trung giới thiệu các kiến thức nền tảng cần thiết cho việc hiểu và phát triển phương pháp đề xuất, bao gồm tối ưu hóa lồi, tối ưu hóa hỗn hợp số nguyên và lý thuyết đồ thị.
- **Chương 3** tổng hợp và phân tích các công trình nghiên cứu liên quan nhằm làm rõ bối cảnh nghiên cứu, cũng như những hạn chế còn tồn tại của các phương pháp hiện nay.
- **Chương 4** trình bày khuôn khổ GCS và cách xây dựng bài toán MICP được đề xuất. Trên cơ sở đó, chúng tôi đưa ra các kết quả thực nghiệm và phân tích hiệu quả của phương pháp trong nhiều môi trường khác nhau với mức độ phức tạp tăng dần.
- **Chương 5** tổng kết toàn bộ nội dung của luận văn và đề xuất các hướng phát triển trong tương lai.

Chương 2

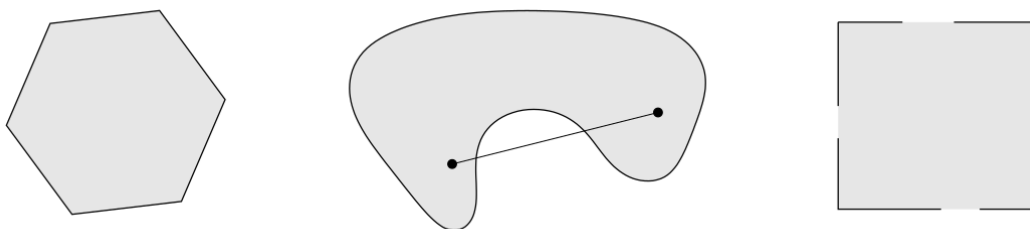
Kiến thức nền tảng

Trong Chương 2, các kiến thức nền tảng trong đề tài được trình bày.

2.1 Giải tích Lồi và Tối ưu hóa

2.1.1 Tập Lồi (Convex Sets)

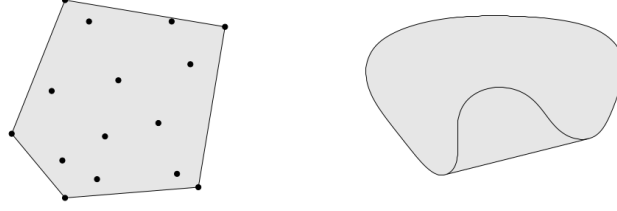
Trước khi thảo luận về các bài toán tối ưu hóa, chúng ta cần thiết lập các đối tượng hình học cơ bản được sử dụng trong công trình này. Một tập hợp $C \subseteq \mathbb{R}^n$ được định nghĩa là *lồi* (convex) nếu, với bất kỳ hai điểm nào thuộc tập hợp, đoạn thẳng nối chúng nằm trọn vẹn bên trong tập hợp đó.



Hình 2.1: Các ví dụ về tập lồi và không lồi. Trái: Hình lục giác (bao gồm cả biên) là tập lồi. Giữa: Tập hình quả thận là không lồi, vì đoạn thẳng nối hai điểm đi ra ngoài tập hợp. Phải: Hình vuông chỉ chứa một phần biên không phải là tập lồi.[6]

Gắn liền mật thiết với tính lồi là khái niệm **bao lồi** (convex hull). Bao lồi

của một tập hợp các điểm $\{x_1, x_2, \dots, x_m\} \subseteq \mathbb{R}^n$ là tập lồi nhỏ nhất chứa các điểm đó, được định nghĩa một cách hình thức là tập hợp của tất cả các tổ hợp lồi (convex combinations):



Hình 2.2: Bao lồi của một tập hợp điểm (phần được tô đậm).[6]

Trong bối cảnh quy hoạch chuyển động và luận văn này, chúng tôi tập trung chủ yếu vào hai loại tập lồi cụ thể:

- **Đa diện (Polyhedra):** Được định nghĩa là giao của một số hữu hạn các nửa không gian (halfspaces), biểu diễn dưới dạng $\mathcal{P} = \{x \in \mathbb{R}^n \mid Ax \leq b\}$. Khi một đa diện bị chặn (bounded), nó được gọi là một *khối đa diện* (polytope).
- **Ellipsoid:** Được định nghĩa là ảnh của một hình cầu đơn vị qua một phép biến đổi afin (affine transformation). Chúng thường được biểu diễn dưới dạng $\mathcal{E} = \{x \in \mathbb{R}^n \mid (x - c)^\top Q^{-1}(x - c) \leq 1\}$, trong đó Q là một ma trận xác định dương ($Q \succ 0$).

2.1.2 Nón Lồi và Đồng nhất hóa (Convex Cones and Homogenization)

Một tập con $\mathcal{K} \subseteq \mathbb{R}^n$ được gọi là *nón* (cone) nếu nó đóng dưới phép nhân vô hướng dương (tức là, $\theta x \in \mathcal{K}$ với mọi $x \in \mathcal{K}, \theta \geq 0$). Một tập hợp được gọi là *nón lồi* (convex cone) nếu nó vừa là tập lồi vừa là nón. Hai khái niệm quan trọng liên quan đến nón lồi là cốt yếu cho công trình này: nón phối cảnh (được xây dựng thông qua quá trình đồng nhất hóa) và nón đối ngẫu.

2.1.2.1 Nón Phối cảnh và Đồng nhất hóa (Perspective Cones and Homogenization)

Đồng nhất hóa (Homogenization) là một kỹ thuật mạnh mẽ trong giải tích lồi, được sử dụng để biến đổi một tập lồi tổng quát thành một nón trong không gian có số chiều cao hơn. Quá trình này thực chất là “nâng” (lifts) một tập $\mathcal{S} \subseteq \mathbb{R}^n$ vào không gian $\mathbb{R}^{n+1}$ bằng cách giới thiệu thêm một chiều phụ y . Tập đồng nhất hóa $\tilde{\mathcal{S}}$ được định nghĩa là:

$$\tilde{\mathcal{S}} := \{(x, y) \in \mathbb{R}^{n+1} \mid y > 0, x \in y\mathcal{S}\}. \quad (2.1)$$

Điểm mấu chốt là $\tilde{\mathcal{S}}$ là tập lồi khi và chỉ khi tập gốc $\mathcal{S}$ là lồi. Phép biến đổi này cho phép biểu diễn các tổ hợp lồi bên trong một cấu trúc nón, điều này đặc biệt hữu ích để chuyển đổi các ràng buộc afin sang dạng nón trong tối ưu hóa.

Khi một tập lồi $\mathcal{C}$ được mô tả dưới dạng nón tiêu chuẩn, việc đồng nhất hóa nó về mặt tính toán là rất trực tiếp. Quan sát rằng với $y > 0$, tập $y\mathcal{C}$ thỏa mãn $\{yx \mid Ax + b \in \mathcal{K}\} = \{x' \mid Ax' + by \in \mathcal{K}\}$, ta thu được *nón phối cảnh* (perspective cone):

$$\tilde{\mathcal{C}} = \{(x, y) \in \mathbb{R}^{n+1} \mid y > 0, Ax + by \in \mathcal{K}\}. \quad (2.2)$$

Một ý nghĩa thực tiễn quan trọng nằm ở bao đóng (closure) của tập này, $\text{cl}(\tilde{\mathcal{C}}) = \{(x, y) \mid y \geq 0, Ax + by \in \mathcal{K}\}$. Bao đóng này mở rộng định nghĩa để bao gồm cả biên nơi $y = 0$, điều này rất quan trọng trong việc mô hình hóa các ràng buộc logic trong quy hoạch nguyên-hỗn hợp.

Đối với một tập compact $\mathcal{C}$, khi vô hướng y tiến dần về 0, tập tỉ lệ $y\mathcal{C}$ sẽ co liên tục về gốc tọa độ. Do đó, trong giới hạn khi $y = 0$, điều kiện $(x, 0) \in \text{cl}(\tilde{\mathcal{C}})$ ngụ ý rằng x bắt buộc phải là vector không (giả sử $\mathcal{C}$ chứa gốc tọa độ và bị chặn). Hành vi hình học này hoạt động hiệu quả như một công tắc logic (logical switch):

- Khi $y > 0$, biến x bị ràng buộc trong phiên bản tỉ lệ của tập $\mathcal{C}$.
- Khi $y = 0$, biến x bị ép về gốc tọa độ ($x = 0$).

Tính chất này được khai thác trực tiếp trong ứng dụng của chúng tôi đối với Bài toán Đường đi Ngắn nhất trên Đồ thị các Tập Lồi (GCS). Trong mô hình Quy hoạch Lồi Nguyên-Hỗn hợp (MICP), biến kích hoạt nhị phân cho một cạnh đóng vai trò là hệ số tỉ lệ y . Khi một cạnh không hoạt động ($y = 0$), các đóng góp trạng thái liên quan bị ép về 0, và chi phí—được mô hình hóa qua hàm phối cảnh—sẽ tự nhiên triệt tiêu. Hàm phối cảnh $\tilde{\ell}_e$ được định nghĩa để trả về 0 khi cả $y = 0$ và các biến trạng thái bằng 0, đảm bảo rằng các thành phần không hoạt động sẽ không phát sinh chi phí trong hàm mục tiêu tối ưu.

2.1.2.2 Nón Đối ngẫu (Dual Cones)

Khái niệm thiết yếu thứ hai là *nón đối ngẫu*. Đối với một nón lồi $\mathcal{K} \subseteq \mathbb{R}^n$, nón đối ngẫu $\mathcal{K}^*$ bao gồm tất cả các vector tạo thành một tích vô hướng không âm với mọi vector trong $\mathcal{K}$:

$$\mathcal{K}^* := \{y \in \mathbb{R}^n \mid x^\top y \geq 0, \forall x \in \mathcal{K}\}. \quad (2.3)$$

Một tính chất cơ bản là $\mathcal{K}^*$ luôn luôn lồi, bất kể $\mathcal{K}$ có lồi hay không. Chúng ta thường gặp ba lớp nón cụ thể, tất cả đều là *tự đối ngẫu* (tức là $\mathcal{K}^* = \mathcal{K}$):

1. **Orthant không âm** (Non-negative orthant): $\mathbb{R}_+^n = \{x \in \mathbb{R}^n \mid x_i \geq 0\}$.
2. **Nón bậc hai** (Second-order cone hay Lorentz cone): $\mathcal{Q}^n = \{(t, x) \in \mathbb{R} \times \mathbb{R}^{n-1} \mid \|x\|_2 \leq t\}$.
3. **Nón bán xác định dương** (Positive semidefinite cone): $\mathcal{S}_+^n = \{X \in \mathcal{S}^n \mid X \succeq 0\}$.

2.1.3 Hàm Lồi (Convex Functions)

Một hàm số $f : \mathcal{D} \rightarrow \mathbb{R}$ là *lồi* (convex) nếu miền xác định $\mathcal{D} \subseteq \mathbb{R}^n$ là một tập lồi và nếu, với mọi $x, y \in \mathcal{D}$ và $\theta \in [0, 1]$, bất đẳng thức sau được thỏa mãn:

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y). \quad (2.4)$$

Về mặt hình học, đoạn thẳng nối bất kỳ hai điểm nào trên đồ thị của hàm số đều nằm phía trên hoặc thuộc đồ thị đó. Một hàm số được gọi là *lồi chặt* (strictly convex) nếu bất đẳng thức này xảy ra dấu nhỏ hơn hẳn với $x \neq y$ và $\theta \in (0, 1)$. Ngược lại, f là *lõm* (concave) nếu $-f$ là lồi.

Tính chất giải tích quan trọng nhất của hàm lồi trong tối ưu hóa là **bất kỳ cực tiểu địa phương nào cũng được đảm bảo là cực tiểu toàn cục**.

2.1.4 Các Bài toán Tối ưu hóa (Optimization Problems)

2.1.4.1 Quy hoạch Lồi (Convex Optimization)

Một **Chương trình Lồi (Convex Program - CP)** được thiết lập như sau:

$$\text{minimize} \quad f(x) \quad (2.5a)$$

$$\text{subject to} \quad x \in \mathcal{C}, \quad (2.5b)$$

trong đó hàm mục tiêu f và tập chấp nhận được (feasible set) $\mathcal{C}$ đều là lồi. Một điểm x^{opt} là một *ng nghiệm tối ưu* nếu nó thuộc tập chấp nhận được và $f(x^{\text{opt}}) \leq f(x)$ với mọi $x \in \mathcal{C}$.

Mặc dù tính lồi về mặt lý thuyết không đảm bảo tính hiệu quả một cách tuyệt đối (ví dụ, kiểm tra tính không âm của đa thức là NP-khó mặc dù

tập hợp này là một nón lồi), nhưng trong phạm vi luận văn này, chúng tôi sử dụng thuật ngữ ‘bài toán tối ưu lồi’ gần như đồng nghĩa với ‘bài toán tối ưu giải được một cách hiệu quả’. Phần lớn các bài toán CP thực tế có thể được giải một cách đáng tin cậy bằng các phương pháp điểm trong (interior-point methods).

2.1.4.2 Quy hoạch Nón (Conic Optimization)

Một **Chương trình Nón (Conic Program - KP)** là một lớp con của CP với hàm mục tiêu tuyến tính và các ràng buộc được định nghĩa bởi một nón lồi $\mathcal{K}$:

$$\text{minimize } c^\top x \quad (2.6a)$$

$$\text{subject to } Ax + b \in \mathcal{K}. \quad (2.6b)$$

Các lớp cụ thể của KP bao gồm:

1. **Quy hoạch Tuyến tính (LP):** $\mathcal{K}$ là orthant không âm.
2. **Quy hoạch Nón Bậc hai (SOCP):** $\mathcal{K}$ là tích của các nón bậc hai.
3. **Quy hoạch Bán xác định (SDP):** $\mathcal{K}$ là nón bán xác định.

Các lớp này đại diện cho một hệ thống phân cấp về khả năng mô hình hóa: $LP \subset SOCP \subset SDP$. Một lớp phổ biến khác là **Quy hoạch Toàn phương (Quadratic Program - QP)**, có hàm mục tiêu bậc hai và các ràng buộc đa diện. Mặc dù mọi bài toán QP đều có thể được thiết lập dưới dạng SOCP, nhưng các thuật toán tập hoạt động (active-set algorithms) chuyên biệt cho QP thường hiệu quả hơn.

2.1.4.3 Quy hoạch Nguyên-Hỗn hợp (Mixed-Integer Programs)

Quy hoạch Nguyên-Hỗn hợp (Mixed-Integer Programs - MIPs) tổng quát hóa bài toán tối ưu bằng cách phân hoạch các biến thành vector

liên tục $x \in \mathbb{R}^n$ và vector nguyên $z \in \mathbb{Z}^m$:

$$\text{minimize} \quad f(x, z) \tag{2.7a}$$

$$\text{subject to} \quad (x, z) \in \mathcal{C}, \tag{2.7b}$$

$$z \in \mathbb{Z}^m. \tag{2.7c}$$

Ràng buộc nguyên làm cho tập chấp nhận được trở nên rời rạc và không lồi, thường khiến các bài toán MIP trở thành NP-khó. Các phương pháp tiêu chuẩn dựa trên tính khả vi không thể áp dụng được.

Để giải quyết MIP, phương pháp **nới lỏng** (relaxation) thường được sử dụng (ví dụ: thay thế $z_j \in \{0, 1\}$ bằng $0 \leq z_j \leq 1$). Giá trị tối ưu của bài toán nới lỏng cung cấp một cận dưới cho nghiệm của MIP. Nếu nghiệm nới lỏng không thỏa mãn tính nguyên, các thuật toán như **Nhánh và Cận** (Branch-and-Bound) sẽ được sử dụng. Phương pháp này phân hoạch không gian tìm kiếm thành các vùng nhỏ hơn (các nhánh) một cách có hệ thống để loại bỏ các giá trị phân số, từ đó duyệt cây tìm kiếm để tìm ra tối ưu toàn cục.

2.2 Lý thuyết Đồ thị và Bài toán Đường đi Ngắn nhất

2.2.1 Cơ sở Lý thuyết Đồ thị

2.2.1.1 Các định nghĩa và khái niệm cơ bản

Một cách hình thức, một đồ thị được định nghĩa là một cặp có thứ tự $G = (\mathcal{V}, \mathcal{E})$, bao gồm một tập các đỉnh $\mathcal{V}$ và một tập các cạnh $\mathcal{E}$, trong đó mỗi cạnh nối một cặp đỉnh. Việc phân loại cấu trúc của đồ thị phụ thuộc vào tính có thứ tự của các cặp này: các cặp không có thứ tự $\{u, w\}$ biểu diễn đồ thị vô hướng, trong khi các cặp có thứ tự (u, w) đặc trưng cho đồ

thị có hướng. Trong bối cảnh tối ưu hóa, đồ thị thường được mở rộng bằng một hàm chi phí $c : \mathcal{E} \rightarrow \mathbb{R}$, gán một trọng số thực cho mỗi cạnh.

Các tính chất tô pô quan trọng bao gồm bậc của đỉnh, là số cạnh liên thuộc với đỉnh đó; trong đồ thị có hướng, khái niệm này được phân biệt thành bậc vào ($\deg^-$) và bậc ra ($\deg^+$). Tính liên thông được mô tả thông qua các đường đi—là các dãy hữu hạn gồm các đỉnh và cạnh phân biệt—trong đó độ dài đường đi được xác định bằng tổng chi phí của các cạnh cấu thành. Một chu trình được định nghĩa là một đường đi bắt đầu và kết thúc tại cùng một đỉnh. Các bài toán tối ưu thường được xét trên một miền tìm kiếm được xác định bởi các đồ thị con $H = (W, F)$, hình thành từ các tập con của đỉnh $W \subseteq \mathcal{V}$ và các cạnh $F \subseteq \mathcal{E}$, đồng thời bảo toàn các quan hệ liên thuộc.

2.2.1.2 Tối ưu hóa tổ hợp trên đồ thị

Các bài toán tối ưu hóa tổ hợp trên đồ thị thường nhằm tìm một cấu trúc con sao cho hàm mục tiêu đạt giá trị nhỏ nhất, đồng thời thỏa mãn các ràng buộc hợp lệ. Với một đồ thị có trọng số $G = (\mathcal{V}, \mathcal{E})$ và một tập các đồ thị con khả thi $\mathcal{H}$, bài toán là xác định một đồ thị con $H = (W, F) \in \mathcal{H}$ sao cho tổng trọng số của các thành phần của nó là nhỏ nhất. Bài toán này được biểu diễn hình thức như sau:

$$\text{tối thiểu hóa} \quad \sum_{v \in W} c_v + \sum_{e \in F} c_e \quad (1.1a)$$

$$\text{với điều kiện} \quad H = (W, F) \in \mathcal{H}. \quad (1.1b)$$

Để có thể áp dụng các kỹ thuật lập trình toán học tiêu chuẩn, chẳng hạn như Branch-and-Bound, bài toán được chuyển đổi sang dạng Quy hoạch Tuyến tính Nguyên (Integer Linear Programming – ILP). Quá trình này bao gồm việc tham số hóa không gian tìm kiếm bằng các vectơ liên thuộc $\mathbf{y}^H \in \{0, 1\}^{|\mathcal{V} \cup \mathcal{E}|}$, trong đó các biến nhị phân biểu thị việc một đỉnh hoặc

một cạnh cụ thể có được chọn vào nghiệm hay không. Về mặt hình học, tập các đồ thị con hợp lệ tương ứng với một đa diện $\mathcal{Y}$ trong không gian Euclid. Giao của đa diện này với lưới số nguyên, $\mathcal{Y} \cap \{0, 1\}^{|\mathcal{V} \cup \mathcal{E}|}$, tạo thành các vectơ liên thuộc của các đồ thị con khả thi. Do đó, bài toán tối ưu rời rạc ban đầu được tái biểu diễn thành một bài toán tối ưu tuyến tính trên miền nguyên:

$$\text{tối thiểu hóa} \quad \sum_{v \in \mathcal{V}} c_v y_v + \sum_{e \in \mathcal{E}} c_e y_e \quad (1.2a)$$

$$\text{với điều kiện} \quad \mathbf{y} \in \mathcal{Y} \cap \{0, 1\}^{|\mathcal{V} \cup \mathcal{E}|}. \quad (1.2b)$$

Cách phát biểu này cho phép áp dụng các công cụ của phân tích lỗi, đồng thời đảm bảo rằng nghiệm tối ưu của mô hình đại số tương ứng chính xác với cấu trúc con tối ưu của bài toán đồ thị ban đầu.

2.2.2 Bài toán Đường đi Ngắn nhất (SPP)

2.2.2.1 Định nghĩa và các thuật toán cổ điển

Bài toán Đường đi Ngắn nhất (Shortest Path Problem – SPP) trên một đồ thị có trọng số $G = (\mathcal{V}, \mathcal{E})$ nhằm xác định một đường đi P từ đỉnh nguồn s đến đỉnh đích t sao cho tổng trọng số các cạnh là nhỏ nhất. Nếu P bao gồm một dãy các cạnh $e_1, \dots, e_k$, mục tiêu là tối thiểu hóa $\sum_{i=1}^k c_{e_i}$.

Các phương pháp cổ điển để giải SPP phụ thuộc vào đặc tính của trọng số cạnh. Thuật toán Dijkstra sử dụng chiến lược tham lam và phù hợp với các đồ thị có trọng số không âm ($c_e \geq 0$). Đối với các đồ thị có cạnh mang trọng số âm, miễn là không tồn tại chu trình âm, thuật toán Bellman–Ford được sử dụng, mặc dù có độ phức tạp tính toán cao hơn. Thuật toán Floyd–Warshall mở rộng bài toán này cho trường hợp tìm đường đi ngắn nhất giữa mọi cặp đỉnh, cho phép tồn tại trọng số âm nhưng vẫn nhạy cảm với sự xuất hiện của các chu trình âm.

2.2.2.2 Mô hình dòng mạng cho Bài toán Đường đi Ngắn nhất

Bài toán Đường đi Ngắn nhất trên một đồ thị có hướng $G = (\mathcal{V}, \mathcal{E})$ với chi phí cạnh không âm $c_{uv} \geq 0$ có thể được mô hình hóa một cách chặt chẽ như một bài toán Dòng Chi phí Tối thiểu với một đơn vị dòng. Ta định nghĩa các biến quyết định nhị phân $x_{uv} \in \{0, 1\}$ cho mỗi cạnh $(u, v) \in \mathcal{E}$, trong đó $x_{uv} = 1$ cho biết cạnh đó thuộc đường đi tối ưu. Mô hình tối ưu được phát biểu như sau:

$$\text{minimize} \quad \sum_{(u,v) \in \mathcal{E}} c_{uv} x_{uv} \quad (2.8a)$$

$$\text{subject to} \quad \sum_{v \in \mathcal{V}^+(u)} x_{uv} - \sum_{v \in \mathcal{V}^-(u)} x_{vu} = \begin{cases} 1 & \text{if } u = s \\ -1 & \text{if } u = t \\ 0 & \text{otherwise} \end{cases}, \quad \forall u \in \mathcal{V} \quad (2.8b)$$

$$\sum_{v \in \mathcal{V}^-(u)} x_{vu} \leq 1, \quad \forall u \in \mathcal{V} \setminus \{s, t\} \quad (2.8c)$$

$$x_{uv} \geq 0, \quad \forall (u, v) \in \mathcal{E} \quad (2.8d)$$

Ở đây, $\mathcal{V}^+(u)$ và $\mathcal{V}^-(u)$ lần lượt ký hiệu tập các đỉnh kế tiếp (successors) và các đỉnh tiền nhiệm (predecessors) của đỉnh u . Các ràng buộc trong mô hình này đảm bảo những tính chất cấu trúc cụ thể. Phương trình (4.8b) biểu diễn điều kiện bảo toàn dòng (định luật Kirchhoff), đảm bảo rằng một đơn vị dòng rời khỏi đỉnh nguồn s , đi vào đỉnh đích t , và được bảo toàn tại tất cả các đỉnh trung gian. Ràng buộc (2.8c) áp đặt giới hạn dung lượng nút, giới hạn dòng đi vào mỗi đỉnh trung gian không vượt quá một đơn vị, qua đó đảm bảo cấu trúc đường đi đơn, trong đó không có đỉnh nào bị ghé thăm nhiều hơn một lần. Với chi phí không âm, các chu trình luôn là không tối ưu, do đó không cần thiết phải bổ sung các ràng buộc loại bỏ chu trình một cách tường minh.

Một ưu điểm quan trọng của cách mô hình hóa này nằm ở các tính chất

đại số của ma trận ràng buộc thu được từ (4.8b). Ma trận liên thuộc đỉnh–cạnh này có tính *hoàn toàn đơn mô* (Total Unimodularity – TUM). Theo lý thuyết tối ưu hóa tổ hợp, nếu ma trận ràng buộc là TUM và vectơ vế phải là nguyên, thì mọi nghiệm khả thi cơ sở của bài toán thư giãn Quy hoạch Tuyến tính (LP) đều được đảm bảo là nguyên [7]. Do đó, mặc dù chỉ áp đặt các ràng buộc không âm liên tục trong (2.8d), bài toán vẫn có thể được giải hiệu quả bằng các thuật toán LP tiêu chuẩn để thu được một đường đi nhị phân hợp lệ, mà không cần đến các phương pháp lập trình nguyên có chi phí tính toán cao.

2.3 Sinh quỹ đạo Động lực học (Kinodynamic)

Mục này trình bày các công thức toán học để sinh ra các quỹ đạo khả thi về mặt động lực học (kinodynamically feasible). Khác với lập kế hoạch đường đi hình học, vốn chỉ xem xét các cấu hình không va chạm trong không gian làm việc, lập kế hoạch chuyển động (kinodynamic) tính đến các ràng buộc vi phân của hệ thống, chẳng hạn như các giới hạn về vận tốc, gia tốc và độ giật (jerk). Chúng tôi tận dụng các đường cong Bézier làm cơ chế tham số hóa quỹ đạo, phương pháp này đặc biệt thuận lợi cho khuôn khổ Đồ thị các Tập lỗi (GCS) nhờ vào các tính chất lỗi của nó.

2.3.1 Mô hình hóa Kinodynamic

Xét một hệ thống robot với sự tiến triển trạng thái được chi phối bởi một tập hợp các phương trình vi phân thường (ODEs):

$$\dot{\mathbf{x}}(t) = f(\mathbf{x}(t), \mathbf{u}(t)), \quad \mathbf{x}(t) \in \mathcal{X}, \mathbf{u}(t) \in \mathcal{U} \quad (2.9)$$

trong đó $\mathbf{x}(t) \in \mathbb{R}^n$ đại diện cho vector trạng thái (thường bao gồm vị trí, vận tốc và gia tốc), và $\mathbf{u}(t) \in \mathbb{R}^m$ biểu thị các tín hiệu điều khiển đầu vào. Mục tiêu là tính toán một quỹ đạo $\sigma(t) : [0, T] \rightarrow \mathcal{X}$ sao cho tối thiểu hóa một phiếm hàm chi phí cụ thể trong khi thỏa mãn:

1. **Điều kiện biên:** Các trạng thái khởi đầu và đích đến $\mathbf{x}(0) = \mathbf{x}_{start}$ và $\mathbf{x}(T) = \mathbf{x}_{goal}$.
2. **Ràng buộc an toàn:** $\sigma(t) \in \mathcal{X}_{free}$ với mọi t , đảm bảo tránh va chạm.
3. **Giới hạn động lực học:** Các ràng buộc bất đẳng thức trên các đạo hàm bậc cao nhằm đảm bảo tính khả thi vật lý:

$$|\dot{\sigma}(t)| \leq v_{max}, \quad |\ddot{\sigma}(t)| \leq a_{max}, \quad |\dddot{\sigma}(t)| \leq j_{max} \quad (2.10)$$

2.3.2 Các ràng buộc về độ trơn và tính liên tục

Trong khuôn khổ GCS, một quỹ đạo hoàn chỉnh được cấu thành từ nhiều đoạn Bézier ghép nối (piecewise). Để đảm bảo chuyển động trơn tru qua các ranh giới của các tập lỗi kề nhau, các ràng buộc liên tục phải được áp đặt tại các điểm chuyển tiếp. Gọi $r_A(t)$ được xác định bởi các điểm điều khiển $\{a_0, \dots, a_n\}$ và $r_B(t)$ được xác định bởi $\{b_0, \dots, b_n\}$ là hai đoạn liên tiếp.

- **Liên tục C^0 (Vị trí):** Đảm bảo đường đi được kết nối liền mạch.

$$a_n = b_0 \quad (2.11)$$

- **Liên tục C^1 (Vận tốc):** Đảm bảo vận tốc liên tục, ngăn chặn các xung gia tốc vô hạn.

$$a_n - a_{n-1} = b_1 - b_0 \quad (2.12)$$

- **Liên tục C^2 (Gia tốc):** Đảm bảo việc tác động lực/mô-men xoắn

diễn ra liên tục, điều này rất quan trọng để giảm thiểu hao mòn cơ khí và độ giật (jerk).

$$(a_n - 2a_{n-1} + a_{n-2}) = (b_2 - 2b_1 + b_0) \quad (2.13)$$

Bằng cách áp đặt các ràng buộc đẳng thức này tại các cạnh của đồ thị, chúng ta tạo ra một quỹ đạo trơn toàn cục tuân thủ các đặc tính vật lý cơ bản của robot.

2.3.3 Cơ sở lý thuyết về đường cong Bézier

Để giải quyết bài toán lập kế hoạch kinodynamic bằng phương pháp số, việc tham số hóa các hàm quỹ đạo thông qua một số hữu hạn các biến quyết định là rất cần thiết. Để đạt được mục đích này, chúng tôi sử dụng các đường cong Bézier. Phần này nhắc lại định nghĩa và các tính chất cơ bản của họ đường cong này. Một đường cong Bézier được xây dựng dựa trên các đa thức Bernstein. Đa thức Bernstein thứ k bậc d , với $k = 0, \dots, d$, được định nghĩa là:

$$\beta_{k,d}(s) := \binom{d}{k} s^k (1-s)^{d-k}, \quad s \in [0, 1] \quad (2.14)$$

Lưu ý rằng các đa thức Bernstein bậc d luôn không âm và theo định lý nhị thức, tổng của chúng bằng 1. Do đó, với mỗi $s \in [0, 1]$ cố định, các vô hướng $\{\beta_{k,d}(s)\}_{k=0}^d$ có thể được xem như các hệ số của một tổ hợp lồi. Các đường cong Bézier thu được bằng cách sử dụng các hệ số này để tổ hợp một tập hợp gồm $d+1$ điểm điều khiển $\gamma_k \in \mathbb{R}^n$ đã cho:

$$\gamma(s) := \sum_{k=0}^d \beta_{k,d}(s) \gamma_k \quad (2.15)$$

Có thể dễ dàng kiểm chứng rằng các đường cong Bézier sở hữu các tính chất sau đây, vốn đóng vai trò thiết yếu đối với khuôn khổ GCS:

- **Giá trị điểm cuối (Endpoint values):** Đường cong γ bắt đầu tại điểm điều khiển đầu tiên và kết thúc tại điểm điều khiển cuối cùng:

$$\gamma(0) = \gamma_0 \quad \text{và} \quad \gamma(1) = \gamma_d \quad (2.16)$$

Tính chất này rất cần thiết cho việc thực thi các ràng buộc liên tục (C^0) khi ghép nối nhiều đoạn Bézier lại với nhau trong đồ thị.

- **Tính chất bao lồi (Convex hull property):** Đường cong γ nằm hoàn toàn bên trong bao lồi của các điểm điều khiển của nó:

$$\gamma(s) \in \text{conv}(\{\gamma_k\}_{k=0}^d), \quad \forall s \in [0, 1] \quad (2.17)$$

Trong bối cảnh của GCS, nếu một vùng lồi X_v đại diện cho một tập an toàn, việc đảm bảo tránh va chạm được quy về việc ràng buộc tất cả các điểm điều khiển phải nằm trong tập hợp đó:

$$\gamma_k \in X_v, \quad \forall k \in \{0, \dots, d\} \implies \gamma(s) \in X_v \quad (2.18)$$

Điều này cho phép chúng ta thực thi các ràng buộc an toàn liên tục thông qua một tập hợp hữu hạn các ràng buộc bao hàm tuyến tính.

- **Đạo hàm (Hodograph):** Đạo hàm $\dot{\gamma}$ của đường cong γ cũng là một đường cong Bézier, nhưng có bậc là $d-1$, với các điểm điều khiển được xác định bởi sai phân của các điểm kề nhau:

$$\dot{\gamma}(s) = \sum_{k=0}^{d-1} \beta_{k,d-1}(s) \dot{\gamma}_k \quad (2.19)$$

trong đó $\dot{\gamma}_k = d(\gamma_{k+1} - \gamma_k)$ với $k = 0, \dots, d-1$. Tương tự, các đạo hàm bậc cao hơn (gia tốc, độ giật) có thể được biểu diễn dưới dạng tổ hợp tuyến tính của các điểm điều khiển. Do đó, các ràng buộc động lực học (ví dụ: giới hạn vận tốc $|\dot{\gamma}(s)| \leq v_{max}$) có thể được áp đặt dưới

dạng các ràng buộc tuyến tính trên sai phân giữa các điểm điều khiển kế nhau, qua đó bảo toàn tính lồi của bài toán tối ưu hóa.

- **Tích phân của hàm lồi:** Đối với một hàm lồi $f : \mathbb{R}^n \rightarrow \mathbb{R}$ (thường đại diện cho hàm chi phí trong lập kế hoạch chuyển động, chẳng hạn như năng lượng hoặc độ dài đường đi), tích phân dọc theo đường cong bị chặn bởi tổ hợp lồi của các giá trị hàm tại các điểm điều khiển:

$$\int_0^1 f(\gamma(s))ds \leq \frac{1}{d+1} \sum_{k=0}^d f(\gamma_k) \quad (2.20)$$

Bất đẳng thức này cung cấp một cận trên lồi cho chi phí tích phân, tạo điều kiện thuận lợi cho việc tối ưu hóa hiệu quả trong khuôn khổ bài toán quy hoạch lồi hỗn hợp số nguyên (mixed-integer convex programming) của GCS.

2.4 Phân hoạch Lồi

Phần này cung cấp tổng quan về phân hoạch lồi, một kỹ thuật cơ bản được sử dụng trong hình học tính toán và mô phỏng vật lý.

2.4.1 Định nghĩa

Phân hoạch lồi (Convex Decomposition) là quá trình chia một hình học phức tạp (ví dụ: đa giác hoặc đa diện không lồi) thành một tập hợp các **vùng con lồi** sao cho hợp của các vùng này khôi phục lại toàn bộ hình ban đầu và các vùng con không chồng lấn nhau, ngoại trừ biên chung.

Về mặt hình thức, với một miền hình học $P \subset \mathbb{R}^n$, một *phân hoạch lồi* của P là tập hợp $\{P_1, P_2, \dots, P_k\}$ sao cho:

$$P = \bigcup_{i=1}^k P_i, \quad P_i \cap P_j = \partial P_i \cap \partial P_j \text{ với mọi } i \neq j,$$

và mỗi P_i là một tập lồi. Mục tiêu là tìm tập phân hoạch này như là số lượng các vùng lồi tối thiểu, giảm thiểu phần chồng chéo hay chất lượng của vùng lồi được tạo ra (tránh quá "mảnh" hoặc "dài").

2.4.2 Phân loại Phương pháp Phân hoạch Lồi

Trong lĩnh vực hình học tính toán và hoạch định chuyển động, các phương pháp phân hoạch lồi thường được chia thành hai loại chính: **Phân hoạch Lồi Chính xác (Exact Convex Decomposition – ECD)** và **Phân hoạch Lồi Xấp xỉ (Approximate Convex Decomposition – ACD)**. Sự khác biệt giữa hai loại này phản ánh sự đánh đổi cơ bản giữa *tính chính xác hình học* và *hiệu quả tính toán*.

2.4.2.1 Phân hoạch Lồi Chính xác (Exact Convex Decomposition – ECD)

Phân hoạch Lồi Chính xác (ECD) là quá trình chia một hình dạng không lồi thành các **thành phần con hoàn toàn lồi**, không chồng lấn, sao cho hợp của chúng tái tạo lại chính xác hình dạng ban đầu. Mục tiêu thường là tối thiểu hóa số lượng vùng lồi.

Tuy nhiên, ECD gặp hai thách thức lớn:

- **Độ phức tạp tính toán cao:** Bài toán tìm phân hoạch lồi tối thiểu đã được chứng minh là NP-hard đối với đa giác có lỗ, nên không tồn tại thuật toán hiệu quả cho các trường hợp tổng quát.
- **Tính khả dụng hạn chế:** Ngay cả khi tính toán được, ECD thường tạo ra số lượng vùng rất lớn, khiến việc lưu trữ, xử lý và tối ưu hóa tiếp theo trở nên tốn kém.

Do đó, các phương pháp ECD chủ yếu mang tính lý thuyết, dùng làm chuẩn so sánh, trong khi hầu hết các ứng dụng thực tế trong robot và mô phỏng đều dựa vào các phương pháp xấp xỉ.

2.4.2.2 Phân hoạch Lỗi Xấp xỉ (Approximate Convex Decomposition – ACD)

Trước những giới hạn của ECD, các phương pháp **Phân hoạch Lỗi Xấp xỉ (ACD)** được phát triển để đạt sự cân bằng giữa độ chính xác và hiệu quả. ACD cho phép các thành phần không hoàn toàn lỗi, mà chỉ “*gần lỗi*” trong một mức dung sai lỗi τ do người dùng xác định hoặc chỉ bao phủ một phần không gian tự do. Triết lý cốt lõi của ACD là chấp nhận một **mức độ lỗi nhỏ có kiểm soát để đổi lấy**:

- **Hiệu quả tính toán cao hơn:** Các thuật toán ACD, chẳng hạn như phương pháp của Lien và Amato, chạy nhanh hơn nhiều so với ECD. [8]
- **Giảm mạnh số lượng vùng:** Cho phép biểu diễn hình dạng với ít vùng lỗi hơn, đơn giản hơn và dễ xử lý hơn.
- **Kiểm soát mức độ chi tiết:** Một số thuật toán ACD như phương pháp của Lien và Amato cho phép người dùng điều chỉnh tham số lỗi τ để cân bằng giữa độ chính xác và số lượng vùng lỗi. Hay đối với phương pháp Visibility Clique Cover (VCC), nó cho phép người dùng chọn chỉ số để thể hiện mức độ bao phủ của vùng lỗi so với không gian tự do ban đầu. [9]

Chương 3

Related works

...

3.1 Sampling-Based Methods

3.2 Discussion

In this chapter, we have seen three distinct approaches to enhance...

Chương 4

Phương pháp đề suất

...

4.1 Phân hoạch không gian an toàn

Thay vì tiếp cận bài toán tránh vật cản theo cách truyền thống là mô tả các ràng buộc không lỗi dựa trên bề mặt vật cản, nhóm tác giả đề xuất một phương pháp chuyển đổi không gian cấu hình tự do (collision-free configuration space) thành hợp của một tập hợp hữu hạn các vùng lỗi bị chặn (bounded convex regions). Trong mô hình này, các vùng lỗi có thể chồng lấn lên nhau, và bài toán hoạch định chuyển động được phát biểu lại dưới dạng một chương trình quy hoạch lỗi nguyên hỗn hợp (Mixed-Integer Convex Program - MICP). Cụ thể, các biến nguyên được sử dụng để gán từng đoạn quỹ đạo đa thức (polynomial trajectory segments) vào các vùng lỗi an toàn cụ thể này. Phương pháp này mang lại lợi thế lớn về mặt tính toán vì số lượng biến nguyên chỉ phụ thuộc vào số lượng vùng an toàn được tạo ra (sử dụng thuật toán IRIS) thay vì phụ thuộc vào số lượng mặt của vật cản, giúp giải quyết hiệu quả các môi trường phức tạp với hàng trăm vật cản. Hơn nữa, việc ràng buộc quỹ đạo nằm trọn trong các tập lỗi đảm bảo an toàn tuyệt đối cho toàn bộ hành trình liên tục, khắc phục nhược điểm "cắt góc" (corner cutting) thường gặp ở các phương pháp chỉ kiểm tra

va chạm tại các điểm lấy mẫu rời rạc.

4.2 Các Phương pháp Cốt lõi cho Phân hoạch Không gian Tự do

Để giải quyết thách thức về tính toán của phân hoạch tối ưu, một số các phương pháp đã được phát triển. Dưới đây là một số phương pháp được dùng trong bài báo cáo này:

4.2.1 Phân hoạch Lỗi Xấp xỉ (ACD) của Đa giác

Phương pháp Approximate Convex Decomposition (ACD) giới thiệu một cách tiếp cận khác biệt cơ bản để quản lý sự phức tạp. Thay vì yêu cầu sự lồi hoàn hảo, nó cho phép các thành phần "lồi xấp xỉ" trong một dung sai do người dùng xác định, τ . [8] Điều này thường dẫn đến các phân hoạch có ít thành phần hơn nhiều và phù hợp hơn với các đặc điểm cấu trúc nổi bật của đối tượng.

4.2.1.1 Các Khái niệm Cơ bản trong Phân rã Lồi

Phần này trình bày các khái niệm hình học cơ bản được sử dụng trong quá trình phân rã đa giác không lồi thành các vùng lồi. Các định nghĩa này đóng vai trò nền tảng cho các phương pháp đo độ lồi và các thuật toán phân hoạch ở các phần sau.

1. **Bao lồi (Convex Hull – H_P):** Bao lồi của một đa giác P là **tập hợp lồi nhỏ nhất chứa toàn bộ đa giác** đó. Trực quan, có thể hình dung bao lồi như một sợi dây chun được kéo căng bao quanh đa giác. Một đa giác P được gọi là **lồi** nếu và chỉ nếu nó trùng với bao lồi của chính nó, tức là:

$$P = H_P.$$

2. **Notches:** Là các đỉnh của đa giác P **không nằm trên bao lồi** H_P . Về mặt hình học, đây là những đỉnh có **góc trong lớn hơn** 180° . Các Notches biểu hiện các đặc trưng lõm (concave features) trên biên của đa giác.

3. **Cầu (Bridges):** Là các cạnh của bao lồi ∂H_P nối hai đỉnh không liên kề của biên ngoài ∂P_0 . Về mặt toán học, tập hợp các cầu được định nghĩa:

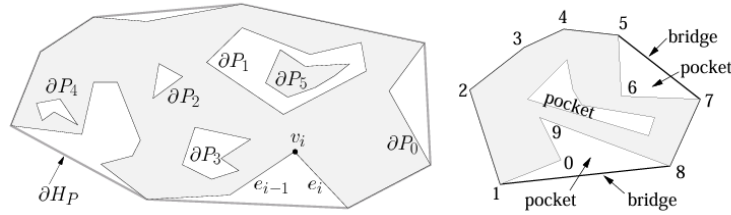
$$\text{BRIDGES}(P) = \partial H_P \setminus \partial P.$$

Mỗi cầu “bắc qua” một vùng lõm của đa giác.

4. **Túi (Pockets):** Là các **chuỗi cạnh liên tiếp** của biên đa giác ∂P không thuộc về vỏ lồi ∂H_P . Về mặt toán học:

$$\text{POCKETS}(P) = \partial P \setminus \partial H_P.$$

Mỗi túi tương ứng với một “vịnh lõm” (concave bay) trên đường biên của đa giác.



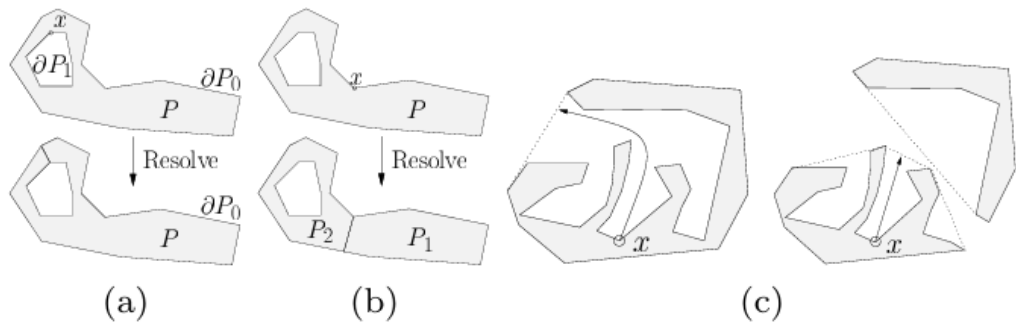
4.2.1.2 Ý tưởng và Chi tiết Thuật toán

Thuật toán là một chiến lược chia để trị đệ quy[8]:

1. Đo độ lõm: Xác định đặc điểm không lồi (một notch hay đỉnh lõm) quan trọng nhất trong đa giác. Đây là điểm có độ lõm lớn nhất. Tùy theo từng trường hợp mà ta có thể sử dụng các thước đo độ lõm khác

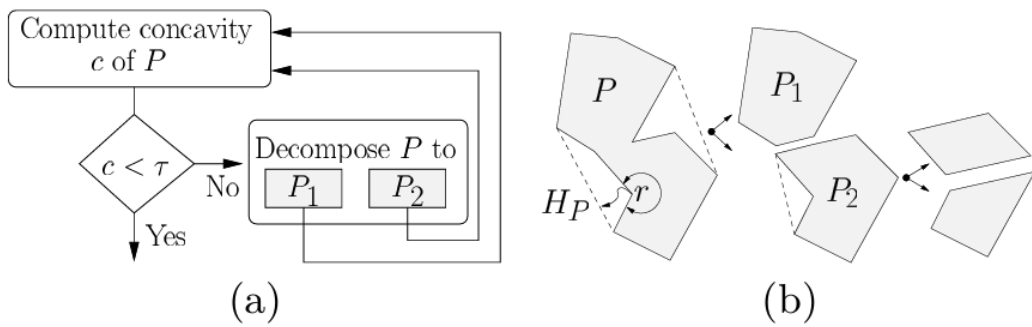
nhau, cách để tính những thước đo độ lõm này sẽ được giới thiệu rõ hơn ở phần 4.2.1.3

2. Kiểm tra Dung sai: Nếu độ lõm tối đa đo được nhỏ hơn τ , quá trình kết thúc đối với thành phần đó.
3. Giải quyết Đỉnh lõm: Nếu độ lõm vượt quá τ , một đường cắt được thêm vào để giải quyết đỉnh lõm, chia đa giác thành hai thành phần mới (hoặc hợp nhất một lỗ).



Hình 4.1: (a) Nếu $x \in \partial P_i > 0$, hàm **Resolve** **gộp** biên ∂P_i vào đa giác P_0 .
(b) Nếu $x \in \partial P_0$, hàm **Resolve** **tách** đa giác P thành hai đa giác P_1 và P_2 .
(c) **Độ lõm** tại điểm x thay đổi sau khi đa giác được phân rã.

4. Độ quy: Quá trình được áp dụng đệ quy cho các thành phần mới.



Hình 4.2: Quá trình đệ quy tiếp tục cho đến khi đạt đến giới hạn độ lõm đầu vào của người dùng τ .

4.2.1.3 Các Thước đo Độ lõm

Chất lượng của phân hoạch phụ thuộc rất nhiều vào **cách đo độ lõm** của các đỉnh trong đa giác không lồi. Các phương pháp đo độ lõm phổ biến bao gồm:

1. Độ lõm Đường thẳng (Straight-Line Concavity – SL-Concavity):

Là khoảng cách Euclid từ một đỉnh trong *túi* (pocket) đến *cầu nối* (bridge) liên kết của nó — thường là một cạnh thuộc bao lồi của đa giác. Phương pháp này rất nhanh, nhưng **không đảm bảo tính đơn điệu**, có thể khiến một số đặc trưng lõm quan trọng bị bỏ sót.



Hình 4.3: Giả sử r là điểm lõm có độ lõm lớn nhất. Sau khi giải quyết (resolve) r , độ lõm của s tăng lên. Nếu $\text{concave}(r) < \tau$, vùng s sẽ không bao giờ được xử lý, ngay cả khi $\text{concave}(s)$ lớn hơn τ .

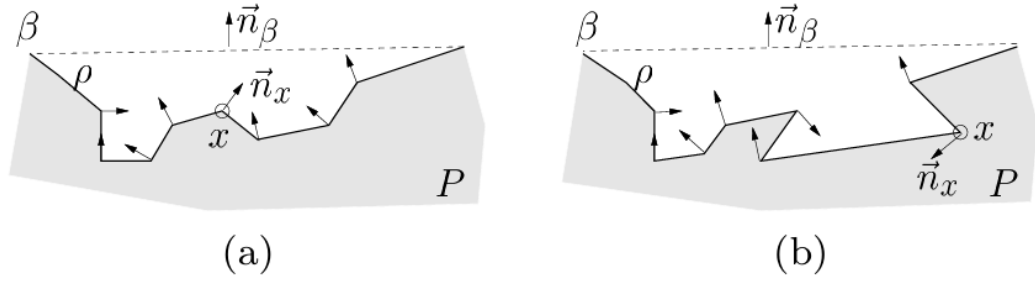
2. Độ lõm Đường đi Ngắn nhất (Shortest Path Concavity – SP-Concavity):

Là **chiều dài đường đi ngắn nhất** từ một đỉnh trong túi đến cầu nối tương ứng, với điều kiện đường đi đó phải nằm hoàn toàn trong túi. Phương pháp này chậm hơn SL-Concavity nhưng **đảm bảo tính đơn điệu**: độ lõm của các đỉnh giảm dần sau mỗi lần phân rã, giúp đảm bảo rằng tất cả các đặc điểm lõm quan trọng cuối cùng sẽ được xử lý. (Xem chi tiết trong Phần ??.)

3. Độ lõm Lai (Hybrid Concavity – H-Concavity):

Một giải pháp trung gian thực tiễn — sử dụng SL-Concavity nhanh làm mặc định và chỉ chuyển sang SP-Concavity khi **phát hiện khả năng không đơn điệu**. Bằng cách thêm bước kiểm tra *nguy cơ tiềm ẩn* bằng cách so sánh n_β và n_i ; nếu tồn tại $n_i \cdot n_\beta < 0$, biên túi bị đảo hướng — dấu

hiệu của túi phức tạp mà SL-Concavity có thể thất bại.



Hình 4.4: Độ lõm SL có thể xử lý được túi trong (a) một cách chính xác vì không có hướng chuẩn nào của các đỉnh trong túi ngược với hướng chuẩn của cầu. Tuy nhiên, túi trong (b) có thể dẫn đến độ lõm giảm không đơn điệu.

4.2.2 Iterative Regional Inflation by Semi-Definite Programming (IRIS)

Thuật toán Iterative Regional Inflation by Semidefinite Programming (IRIS) thay vì cố gắng bao phủ toàn bộ không gian tự do cùng một lúc, nó đặt ra câu hỏi: "Với một điểm 'mầm' (seed) ban đầu, vùng lồi lớn nhất của không gian tự do mà tôi có thể tìm thấy xung quanh nó là gì?". Tức sau khi chạy xong giải thuật output của ta là một vùng lồi lớn nhất có thể của thuật toán từ 1 seed point cho trước.

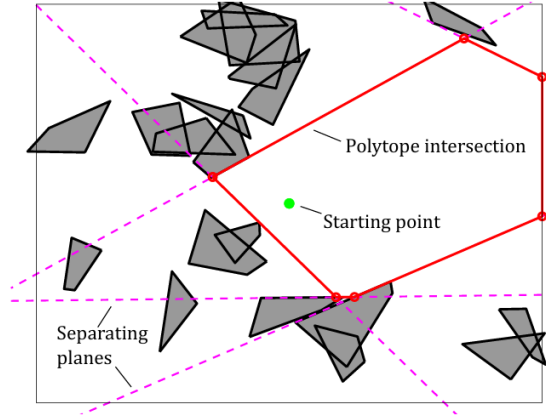
4.2.2.1 Ý tưởng và Chi tiết Thuật toán

IRIS là một quy trình tối ưu hóa lặp đi lặp lại gồm 4 bước:

1. **Khởi tạo:** Thuật toán sẽ tạo ra một hình elip ban đầu là một hình cầu rất nhỏ có tâm chính là điểm mầm (seed point).

2. Tìm Siêu phẳng Phân cách (QP):

Đối với ellipsoid hiện tại, thuật toán tìm điểm gần nhất trên mỗi chướng ngại vật. Sau đó, nó xây dựng các siêu phẳng phân cách ellipsoid khỏi các chướng ngại vật này. Mỗi siêu phẳng tiếp tuyến với



Hình 4.5: Siêu phẳng Phân cách

một phiên bản mở rộng của ellipsoid và đi qua điểm gần nhất trên chướng ngại vật tương ứng. Việc tìm điểm gần nhất trên mỗi chướng ngại vật lỗi được xây dựng như một bài toán Quy hoạch Toàn phương (Quadratic Program - QP). Để giữ cho bài toán tối ưu hóa tiếp theo có quy mô nhỏ, các siêu phẳng thừa (những siêu phẳng không xác định biên của vùng lỗi kết quả) sẽ được loại bỏ.

Bước 2.1. Tìm điểm gần nhất trên obstacle đến hình elip hiện tại
Biến đổi không gian (Từ không gian Ellipsoid sang không gian Quả cầu đơn vị)

Việc tính toán khoảng cách đến một ellipsoid là phức tạp. Tuy nhiên, tính khoảng cách đến một quả cầu đơn vị tại gốc tọa độ thì lại rất đơn giản. Ý tưởng ở đây là biến đổi toàn bộ không gian sao cho ellipsoid trở thành một quả cầu đơn vị. Cụ thể, một ellipsoid E được định nghĩa là ảnh của một quả cầu đơn vị $\tilde{E}$ qua phép biến đổi affine $x = C\tilde{x} + d$, trong đó C là ma trận xác định hình dạng và hướng của ellipsoid, còn d là tâm của nó. Khi đó, ta có thể sử dụng phép biến đổi ngược $\tilde{x} = C^{-1}(x - d)$ để ánh xạ mọi điểm trong không gian ban đầu trở lại "không gian quả cầu đơn vị". Sau phép biến đổi này, ellipsoid E trở thành quả cầu đơn vị $\tilde{E}$ tại gốc tọa độ, và đồng thời mỗi chướng ngại vật l_j trong không gian ban đầu cũng bị biến đổi (méo đi) thành một đối tượng mới $\tilde{l}_j$ trong không gian mới.

Bước 2.2: Tìm điểm gần nhất bằng Quy hoạch Toàn phương (QP)

Bây giờ, bài toán tìm điểm trên chướng ngại vật l_j gần ellipsoid E nhất tương đương với việc tìm điểm trên chướng ngại vật đã biến đổi $\tilde{l}_j$ gần gốc tọa độ nhất.

Bài toán này được xây dựng dưới dạng một bài toán *Quy hoạch Toàn phương* (Quadratic Program - QP):

$$\begin{aligned} & \underset{\tilde{x} \in \mathbb{R}^n, w \in \mathbb{R}^m}{\text{minimize}} && \|\tilde{x}\|^2 \\ & \text{subject to} && \begin{cases} [\tilde{v}_{j,1} \ \tilde{v}_{j,2} \ \cdots \ \tilde{v}_{j,m}] w = \tilde{x}, \\ \sum_{i=1}^m w_i = 1, \\ w_i \geq 0, \quad i = 1, \dots, m. \end{cases} \end{aligned}$$

Về bản chất, ta đang tìm một điểm $\tilde{x}$ là **tổ hợp lồi** của các đỉnh đã biến đổi $\tilde{v}_{j,k}$ sao cho khoảng cách của $\tilde{x}$ đến gốc tọa độ là nhỏ nhất.

Bước 2.3. Xác định mặt phẳng tiếp tuyến của ellipsoid tại điểm tiếp xúc

Sau khi tìm được điểm x^* gần nhất, ta xây dựng mặt phẳng tiếp tuyến với ellipsoid tại điểm đó. Ellipsoid được mô tả bằng biểu thức nghịch đảo:

$$E = \{x \mid (x - d)^\top C^{-1} C^{-\top} (x - d) \leq 1\}.$$

Vector pháp tuyến của ellipsoid tại x^* được xác định bởi gradient:

$$a_j = \nabla_x [(x - d)^\top C^{-1} C^{-\top} (x - d)]_{x=x^*} = 2C^{-1} C^{-\top} (x^* - d).$$

Vì mặt phẳng tiếp tuyến đi qua x^* , ta có:

$$b_j = a_j^\top x^*.$$

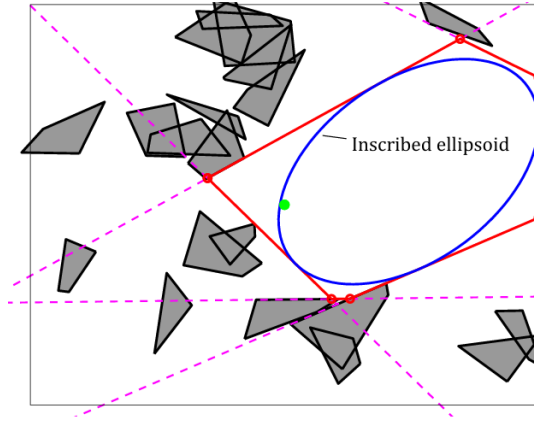
Khi đó, phương trình mặt phẳng tiếp tuyến là:

$$a_j^\top x = b_j,$$

được thêm vào như một **ràng buộc tuyến tính** trong tập khả thi của thuật toán IRIS. Mặt phẳng này đảm bảo ellipsoid mở rộng tối đa nhưng không giao với chướng ngại vật.

Bước 2.4. Lặp lại với tất cả các obstacles trong không gian. Ta sẽ có được một tập các siêu phẳng để tạo thành một vùng lồi đa giác.

3. **Tìm Ellipsoid Nội tiếp Cực đại (SDP):** Giao của các nửa không gian được xác định bởi các siêu phẳng tạo thành một đa diện lồi (polytope). Thuật toán sau đó tìm ellipsoid có thể tích lớn nhất có thể nội tiếp trong đa diện này.



Hình 4.6: Ellipsoid Nội tiếp Cực đại

Bài toán này tìm **ellipsoid có thể tích lớn nhất** nằm gọn trong một đa diện lồi P :

$$P = \{x \in \mathbb{R}^n \mid Ax \leq b\}. \quad (4.1)$$

Ellipsoid được định nghĩa là $\mathcal{E}$, với d là tâm và ma trận C xác định

hình dạng:

$$\mathcal{E} = \{C\tilde{x} + d \mid \|\tilde{x}\|_2 \leq 1\}. \quad (4.2)$$

Thể tích của ellipsoid tỉ lệ với $\det(C)$.

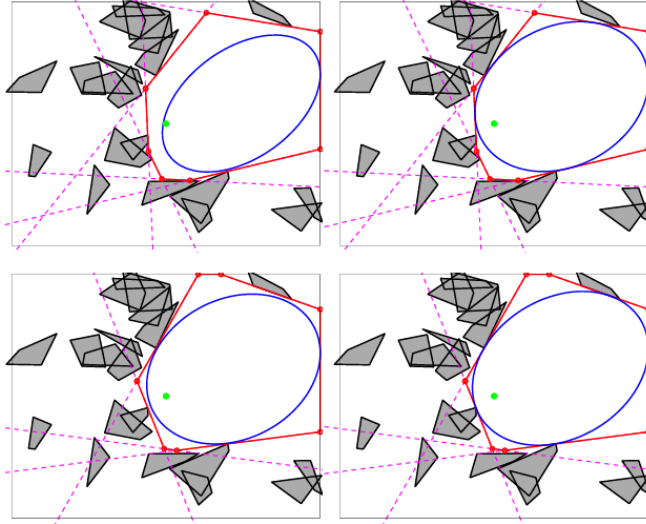
Để tìm ellipsoid lớn nhất nội tiếp trong P , ta giải bài toán tối ưu lồi sau:

$$\begin{aligned} & \underset{C, d}{\text{maximize}} && \log \det C \\ & \text{subject to} && \|a_i^\top C\|_2 + a_i^\top d \leq b_i, \quad \forall i = 1, \dots, N, \\ & && C \succ 0. \end{aligned} \quad (4.3)$$

Mục tiêu: Cực đại hoá $\log \det C$ để tối đa hoá thể tích ellipsoid.

Ràng buộc: Đảm bảo mọi điểm của ellipsoid đều nằm trong đa diện P .

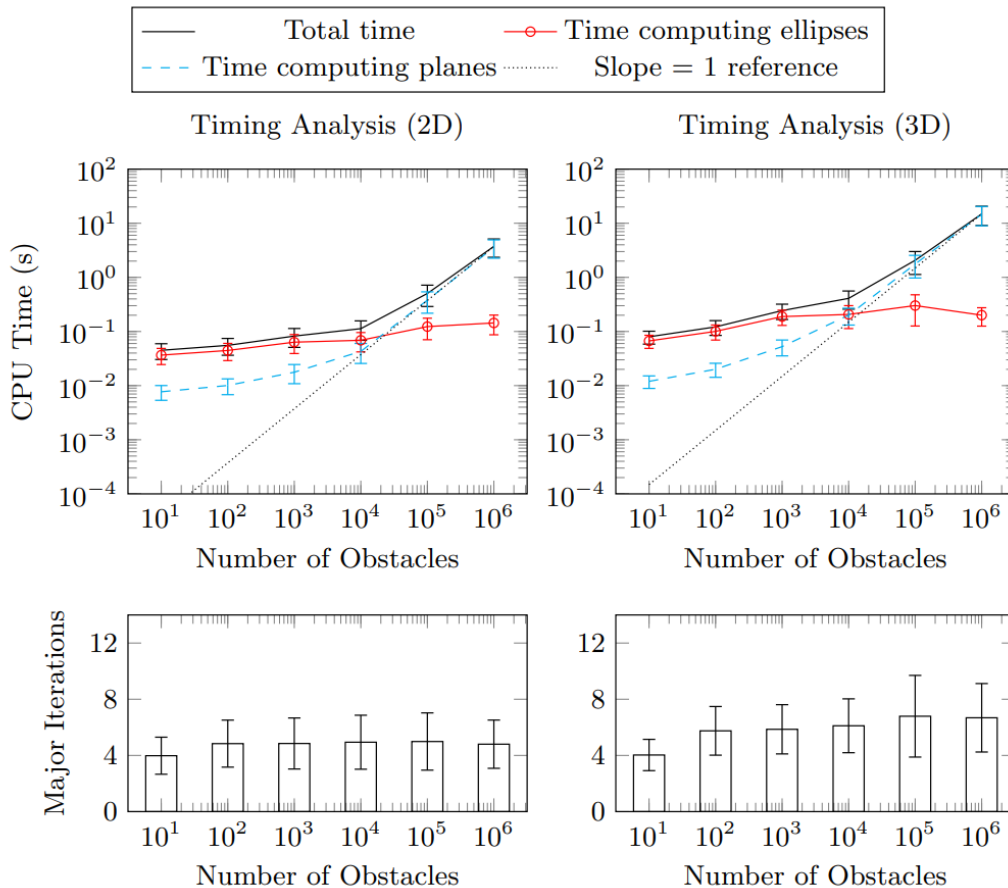
4. **Lặp lại:** Ellipsoid mới, lớn hơn này được sử dụng làm điểm khởi đầu cho vòng lặp tiếp theo. Quá trình này hội tụ khi thể tích của ellipsoid ngừng tăng một cách đáng kể, trả về một đa diện lồi lớn và ellipsoid nội tiếp của nó.



Hình 4.7: Lặp lại đến khi hội tụ

4.2.2.2 Đặc tính của Thuật toán

- Loại thuật toán: IRIS là một phương pháp xấp xỉ để bao phủ toàn bộ không gian tự do. Một lần chạy duy nhất chỉ tạo ra một vùng lõi. Để có được một lớp phủ hoàn chỉnh, cần phải chạy thuật toán nhiều lần với các điểm mầm khác nhau.
- Độ phức tạp thời gian: IRIS thường hội tụ qua 4-8 vòng lặp. Thời gian tính toán của mỗi vòng lặp bằng thời gian tìm các mặt phẳng phân cách cộng với thời gian tính toán ellipses nội tiếp. Trong khi thời gian giải ellipses (SDP) gần như không đổi (constant) nhờ vào việc loại bỏ các siêu phẳng thừa thì thời gian tính các siêu phẳng phân cách lại tăng tuyến tính theo số lượng các obstacles.



Hình 4.8: Độ phức tạp thời gian [10]

- Xử lý Vật cản không lõi:

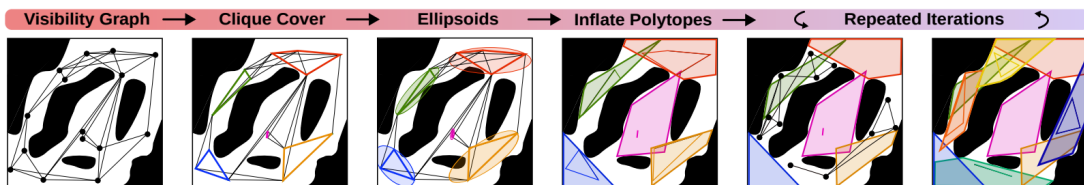
Chướng ngại vật trong không gian work space: Thuật toán IRIS ban đầu giả định các chướng ngại vật là lỗi, đây cũng là một nhược điểm lớn nhất của thuật toán này. Để đảm bảo các obstacles là lỗi, ta thường phải tiền xử lý bằng cách bao lỗi các chướng ngại vật không lỗi[10]. Ngoài ra cách tiếp cận tiêu chuẩn cho các chướng ngại vật không lỗi là phân hoạch chúng thành một tập hợp các mảnh lỗi trước khi chạy IRIS (tức tăng số lượng obstacles của không gian lên). Điều này khả thi vì IRIS có khả năng mở rộng hiệu quả với số lượng lớn chướng ngại vật.

Chướng ngại vật trong không gian cấu hình (Configuration Space): Đối với các robot có động học phức tạp, nơi các chướng ngại vật trong không gian cấu hình được định nghĩa một cách ảm ý và không lỗi, các phần mở rộng như IRIS-NP [11] và C-IRIS [12] được sử dụng.

4.2.3 Visibility Clique Cover (VCC)

Thuật toán Visibility Clique Cover (VCC) giải quyết trực tiếp một hạn chế chính của IRIS: nhu cầu về các điểm mẫu tốt.[9] Việc lấy mẫu ngẫu nhiên là không hiệu quả. VCC tự động hóa quá trình này bằng cách trước tiên tìm hiểu cấu trúc toàn cục của không gian tự do và sau đó đặt các điểm mẫu một cách chiến lược vào các khu vực có khả năng mở rộng lớn.

4.2.3.1 Ý tưởng và Chi tiết Thuật toán



Hình 4.9: Tổng quan về Quy trình VCC[9]

Quy trình VCC như đã thể hiện ở hình 4.9 bao gồm bốn bước chính:

1. Lấy mẫu & Xây dựng Đồ thị Khả kiến (Visibility Graphs): Thuật

toán lấy mẫu n điểm ngẫu nhiên trong không gian cấu hình tự do (C_{free}). Các điểm này tạo thành các đỉnh của một đồ thị khả kiến (visibility graph)¹. Một cạnh tồn tại giữa hai đỉnh nếu đoạn thẳng nối hai điểm tương ứng hoàn toàn nằm trong không gian tự do (tức là không va chạm).

2. Tìm Clique Cover: Thuật toán tìm một tập hợp các clique lớn (đồ thị con đầy đủ) trong đồ thị khả kiến. Ý tưởng cốt lõi là một tập hợp các điểm mà tất cả đều "nhìn thấy" nhau có khả năng nằm trong cùng một vùng lỗi². Điều này được thực hiện một cách tham lam bằng cách giải lặp đi lặp lại bài toán MAXCLIQUE (tìm clique lớn nhất), được xây dựng dưới dạng một bài toán Integer Linear Programming - ILP.
3. Khởi tạo Ellipsoid: Đối với mỗi clique được tìm thấy, thuật toán tính toán ellipsoid có thể tích nhỏ nhất bao quanh tất cả các điểm trong clique đó. Ellipsoid này cung cấp một tâm (điểm mẫu) và các trục chính (hình dạng ban đầu) cho bước lặp đầy tiếp theo.
4. Lấp đầy thành Đa diện: Tâm và hình dạng của mỗi ellipsoid được sử dụng để khởi tạo một vòng lặp duy nhất của thuật toán lấp đầy tương tự IRIS. Điều này hiệu quả hơn IRIS tiêu chuẩn vì ellipsoid ban đầu đã được "thông báo" về hình học cục bộ, thường loại bỏ sự cần thiết của nhiều vòng lặp tốn kém về mặt tính toán.[9]
5. Lấp lại và Hội tụ: Quá trình này được lặp lại cho đến khi đạt được độ phủ đủ bằng cách lấy mẫu mới từ không gian trống còn lại và lặp lại các bước trước đó.

¹ *Visibility graph* là đồ thị trong đó các đỉnh biểu diễn vị trí (hoặc đỉnh chướng ngại vật), và có cạnh nối giữa hai đỉnh nếu đoạn thẳng nối chúng nằm hoàn toàn trong không gian tự do.

² Nếu hai điểm có thể nhìn thấy nhau, điều đó ngụ ý rằng con đường thẳng nhất giữa chúng là an toàn. Đây là khối xây dựng cơ bản để hiểu và xấp xỉ các vùng lỗi trong.

4.3 Đồ thị của các Tập Lồi (Graphs of Convex Sets)

Graph of Convex Sets (GCS) Framework cung cấp một phương pháp luận thống nhất để giải quyết các bài toán tối ưu hóa lai ghép rời rạc-liên tục (hybrid discrete-continuous optimization). Nó đóng vai trò cầu nối hiệu quả giữa tìm kiếm đồ thị tổ hợp (combinatorial graph search) và quy hoạch lồi liên tục (continuous convex programming). Khác với lý thuyết đồ thị cổ điển, GCS tổng quát hóa Bài toán Đường đi Ngắn nhất (SPP) bằng cách liên kết trực tiếp các biến hình học liên tục với tô pô của đồ thị.

4.3.1 Công thức Đường đi Ngắn nhất trong Graph of Convex Sets

Một GCS được định nghĩa là một đồ thị có hướng $G = (V, E)$, trong đó mỗi đỉnh $v \in V$ gắn liền với một biến quyết định liên tục x_v nằm trong một tập lồi compact, khác rỗng $X_v \subset \mathbb{R}^n$. Trong ngữ cảnh lập kế hoạch chuyển động (motion planning), các tập hợp này thường đại diện cho các vùng không va chạm đã được tính toán trước trong không gian cấu hình (configuration space). Các bước chuyển tiếp giữa các đỉnh được quy định bởi các cạnh $e = (u, v) \in E$, áp đặt các ràng buộc lồi phụ trợ $(x_u, x_v) \in X_e$ và được trọng số hóa bởi một hàm chi phí lồi $\ell_e(x_u, x_v)$. Hàm này bắt buộc phải là hàm thực sự (proper), đóng (closed) và lồi, liên kết tường minh các biến liên tục x_u và x_v để mô hình hóa các thước đo chuyển tiếp như khoảng cách Euclid hoặc tiêu hao năng lượng. Cần lưu ý rằng, mặc dù sử dụng thuật ngữ này, chúng tôi không giả định ℓ_e là một metric hợp lệ; các tính chất như tính đối xứng hay bất đẳng thức tam giác không bắt buộc phải thỏa mãn [13]. Trong khi công thức luồng mạng (network flow) cổ điển giải quyết hiệu quả SPP trên các đồ thị rời rạc với chi phí cạnh tĩnh, khung GCS tổng quát

hóa điều này bằng cách đưa vào các biến quyết định liên tục gắn liền với tô pô đồ thị. Trong GCS, chi phí di chuyển qua một cạnh $e = (u, v)$ không phải là một hằng số vô hướng c_{uv} , mà là một hàm lồi $\ell_e(x_u, x_v)$ phụ thuộc vào các trạng thái liên tục $x_u \in X_u$ và $x_v \in X_v$ tại các đỉnh liên thuộc. Mục tiêu của SPP trong GCS là xác định một đường đi p từ đỉnh nguồn s đến đỉnh đích t sao cho tối thiểu hóa tổng chi phí di chuyển qua các cạnh dọc theo đường đi, đồng thời lựa chọn các biến liên tục phù hợp tại mỗi đỉnh. Một cách hình thức, bài toán này được biểu diễn như sau:

$$\min_{p, \{x_v\}_{v \in p}} \sum_{(u,v) \in E_p} \ell_{(u,v)}(x_u, x_v) \quad (4.4)$$

$$\text{s.t. } p \in \mathcal{P}, \quad (4.5)$$

$$x_v \in X_v, \quad \forall v \in p, \quad (4.6)$$

$$(x_u, x_v) \in X_e, \quad \forall e = (u, v) \in E_p \quad (4.7)$$

trong đó $\mathcal{P}$ ký hiệu tập hợp tất cả các đường đi khả thi từ s đến t , và E_p đại diện cho các cạnh nằm trong đường đi p .

Bằng cách áp dụng công thức luồng mạng vào GCS, SPP có thể được biểu diễn dưới dạng bài toán Quy hoạch Phi tuyến Nguyên Hỗn hợp (Mixed-Integer Nonlinear Program - MINLP) như sau:

$$\text{minimize} \quad \sum_{e=(u,v) \in \mathcal{E}} y_e \ell_e(x_u, x_v) \quad (4.8a)$$

$$\text{subject to} \quad \sum_{e \in \mathcal{E}^+(u)} y_e - \sum_{e \in \mathcal{E}^-(u)} y_e = \delta_u, \quad \forall u \in \mathcal{V} \quad (4.8b)$$

$$(x_u, x_v) \in X_e, \quad \forall e = (u, v) \in \mathcal{E} \quad (4.8c)$$

$$x_v \in X_v, \quad \forall v \in \mathcal{V} \quad (4.8d)$$

$$y_e \in \{0, 1\}, \quad \forall e \in \mathcal{E} \quad (4.8e)$$

Trong công thức này, biến nhị phân y_e chỉ thị việc cạnh e có được đưa vào

đường đi được chọn hay không ($y_e = 1$) hoặc không ($y_e = 0$). Ràng buộc bảo toàn luồng (4.8b) đảm bảo rằng các cạnh được chọn tạo thành một đường đi hợp lệ từ đỉnh nguồn s đến đỉnh đích t , trong đó δ_u được định nghĩa là 1 cho nguồn, -1 cho đích, và 0 cho tất cả các đỉnh khác. Các ràng buộc (4.8c) và (4.8d) bắt buộc các biến liên tục tại mỗi đỉnh và dọc theo mỗi cạnh phải tuân thủ các tập lỗi tương ứng của chúng.

4.3.2 Phân tích Độ phức tạp

Độ phức tạp tính toán của SPP trong GCS khác biệt cơ bản so với Bài toán Đường đi Ngắn nhất cổ điển trên đồ thị rời rạc. Trong lý thuyết đồ thị cổ điển, SPP nói chung là NP-hard (ví dụ: bài toán Đường đi Dài nhất hoặc các biến thể liên quan đến chu trình âm). Tuy nhiên, nó trở nên giải được một cách hiệu quả trong thời gian đa thức (polynomial time) dưới hai điều kiện tiên quyết:

1. Đồ thị là **không chu trình** (acyclic) (cho phép giải bằng sắp xếp tô pô).
2. Tất cả chi phí cạnh và đỉnh là **không âm** (giải được bằng thuật toán Dijkstra).

Mệnh đề 1. *Trái ngược hoàn toàn với trường hợp trên, bài toán Đường đi ngắn nhất trong Graph of Convex Sets vẫn là **NP-hard** ngay cả khi đồng thời thỏa mãn hai điều kiện: đồ thị không chu trình và chi phí không âm.*

Độ khó này được thiết lập thông qua việc quy dẫn (reduction) từ bài toán **3SAT** (bài toán thỏa mãn Boolean với các mệnh đề gồm ba biến), vốn được biết đến là NP-đầy đủ (NP-complete). Như đã chứng minh trong Định lý 9.1 của [13], ta có thể xây dựng một thể hiện GCS phân lớp, không chu trình với chi phí không âm, trong đó việc xác định một đường đi hợp lệ tương đương với việc tìm một phép gán thỏa mãn cho công thức 3SAT. Điều này ngụ ý rằng chỉ riêng việc xác định tính *khả thi* (feasibility) của

một SPP trong GCS đã là NP-hard. Các chứng minh khác về tính NP-hard cũng đã được đưa ra cho các trường hợp liên quan đến các tập lỗi chiều thấp (1D và 2D), mặc dù các trường hợp này thường dựa trên các đồ thị có chứa chu trình [14].

4.3.3 Mô hình chi tiết các đỉnh và cạnh

Trong bối cảnh tối ưu quỹ đạo, các thành phần của GCS được định nghĩa một cách chặt chẽ nhằm đảm bảo cả tính an toàn hình học lẫn tính khả thi về động học.

Mỗi đỉnh $v \in V$ tương ứng với một vùng lỗi, không va chạm Q_v trong không gian cấu hình (C-space). Biến quyết định liên tục $x_v \in \mathbb{R}^{n_v}$ gắn với đỉnh v biểu diễn một đoạn quỹ đạo cục bộ hoặc một cấu hình trạng thái. Cần nhấn mạnh rằng chiều n_v là cục bộ theo từng đỉnh, cho phép đồ thị có các đỉnh với số chiều biến khác nhau.

Biến x_v bị ràng buộc trong một tập lỗi compact $\mathcal{X}_v \subset \mathbb{R}^{n_v}$. Đối với bài toán sinh quỹ đạo sử dụng các tham số đa thức từng đoạn (ví dụ như đường cong Bézier), tập $\mathcal{X}_v$ được xây dựng một cách tường minh nhằm đảm bảo quỹ đạo nằm hoàn toàn trong vùng an toàn Q_v và thỏa mãn các giới hạn động lực học. Các ràng buộc xác định $\mathcal{X}_v$ thường có dạng:

$$r_{i,k} \in Q_i, \quad k = 0, \dots, d, \quad (4.9)$$

$$\dot{h}_{i,k} \geq \dot{h}_{\min} = 10^{-6}, \quad k = 0, \dots, d-1, \quad (4.10)$$

$$\dot{r}_{i,k} \in \dot{h}_{i,k} D, \quad k = 0, \dots, d-1, \quad (4.11)$$

$$h_{i,0} \geq 0, \quad h_{i,d} \leq T_{\max}. \quad (4.12)$$

Trong đó, (4.9) đảm bảo các điểm điều khiển không gian $r_{i,k}$ nằm trong vùng an toàn. Phương trình (4.10) đảm bảo tính đơn điệu chặt của hàm co giãn thời gian nhằm duy trì tính nhân quả. Phương trình (4.11) liên kết

đạo hàm không gian và thời gian để thỏa mãn giới hạn vận tốc được xác định bởi tập D , và (4.12) áp đặt các giới hạn lên thời gian thực hiện quỹ đạo.

Một cạnh $e = (u, v) \in E$ xác định sự chuyển tiếp giữa hai đỉnh. Chuyển tiếp này được điều khiển bởi một *tập ràng buộc liên kết* $\mathcal{X}_e \subseteq \mathbb{R}^{n_u+n_v}$. Tập này áp đặt các ràng buộc lên vector ghép (x_u, x_v) , đảm bảo các cặp cấu hình (x_u, x_v) là tương thích với nhau, chẳng hạn như thỏa mãn tính liên tục C^0 , C^1 hoặc bậc cao hơn giữa các đoạn quỹ đạo.

Gắn với mỗi cạnh là một *hàm chi phí liên kết* $f_e : \mathbb{R}^{n_u+n_v} \rightarrow \mathbb{R}$. Đây là một hàm lỗi giá trị vô hướng, đánh giá chi phí dựa trên việc lựa chọn đồng thời x_u và x_v , nhằm mô hình hóa các đại lượng như độ dài đường đi hoặc năng lượng tiêu thụ.

4.3.4 Đặc tả hàm chi phí

Việc lựa chọn hàm chi phí cạnh $\ell_e(x_u, x_v)$ mang tính then chốt, vì nó quy định ý nghĩa vật lý của quỹ đạo tối ưu. Khung làm việc của chúng tôi phân biệt giữa hai mô hình chính: lập kế hoạch đường đi hình học (LinearGCS) và tối ưu hóa quỹ đạo động học (BezierGCS), mỗi mô hình sử dụng các cấu trúc lỗi cụ thể để đảm bảo hiệu quả tính toán.

4.3.4.1 Độ dài Đường đi Hình học

Đối với các bài toán được xác định chặt chẽ bởi các ràng buộc hình học, chẳng hạn như tìm đường đi ngắn nhất qua các vùng lỗi, hàm chi phí được đặc trưng bởi khoảng cách Euclid có trọng số. Bình phương khoảng cách Euclid,

$$\ell_e(x_u, x_v) = \|x_v - x_u\|_2^2,$$

là một hàm bậc hai, phù hợp tự nhiên với các bộ giải Quadratic Programming (QP). Ngoài ra, chuẩn L_2 tiêu chuẩn

$$\ell_e(x_u, x_v) = \|x_v - x_u\|_2$$

có thể được mô hình hóa bằng Quy hoạch Nón Bậc hai (Second-Order Cone Programming - SOCP), giúp bảo toàn tính lồi. Các công thức này đảm bảo rằng chi phí tỷ lệ tuyến tính với khoảng cách Euclid, mang lại một thước đo chính xác về mặt hình học cho độ dài đường đi.

4.3.4.2 Chi phí Quỹ đạo Động lực học

Trong khung lập kế hoạch chuyển động đề xuất sử dụng đường cong Bézier, biến quyết định liên tục x_v được xây dựng để bao hàm cả các điểm điều khiển không gian và thời gian thực hiện của đoạn quỹ đạo. Để quản lý hiệu quả sự đánh đổi giữa tốc độ thực thi và nỗ lực điều khiển (control effort), hàm chi phí được xây dựng như một mục tiêu tổng hợp bao gồm ba thành phần riêng biệt.

Đầu tiên, để ưu tiên tính tối ưu về thời gian, một chi phí tuyến tính được áp dụng cho các biến co giãn thời gian h . Công thức này đảm bảo rằng việc tối thiểu hóa tổng thời gian quỹ đạo vẫn bảo toàn tính lồi của hàm mục tiêu. Chi phí thời gian được biểu diễn là:

$$J_t = w_t T_e, \tag{4.13}$$

trong đó w_t biểu thị hệ số trọng số được gán cho thời gian thực hiện đoạn T_e .

Đồng thời, để phạt nỗ lực điều khiển—cụ thể là tối thiểu hóa tích phân của bình phương vận tốc hoặc gia tốc—khung làm việc tận dụng các tính chất toán học của hàm phối cảnh (perspective functions). Đối với một đoạn quỹ

đạo có thời gian T_e , chi phí năng lượng này được định nghĩa là:

$$J_e = \int_0^{T_e} |\dot{r}(t)|^2 dt. \quad (4.14)$$

Số hạng này hoạt động như một hàm phối cảnh có dạng bậc hai trên tuyến tính (quadratic-over-linear) ($x^T x/y$). Một ưu điểm quan trọng của công thức này là nó lồi đồng thời (jointly convex) đối với cả các điểm điều khiển không gian và biến thời gian. Do đó, điều này cho phép bộ giải tối ưu hóa đường đi hình học và phân bổ thời gian một cách đồng thời trong một khung Quy hoạch Lồi Nguyên Hỗn hợp (Mixed-Integer Convex Programming - MICP) thống nhất.

Cuối cùng, để đảm bảo độ trơn bậc cao, chẳng hạn như tối thiểu hóa độ giật (jerk), hàm chi phí kết hợp điều chuẩn đạo hàm (derivative regularization). Khác với chi phí năng lượng, điều này đạt được bằng cách áp dụng các chi phí bậc hai tiêu chuẩn trực tiếp lên các hệ số của các đạo hàm bậc cao của quỹ đạo. Cách tiếp cận này tránh được các yêu cầu nghiêm ngặt của các công thức phối cảnh trong khi vẫn duy trì thành công cấu trúc lồi tổng thể của bài toán tối ưu hóa.

4.3.5 Xây dựng Bài toán Quy hoạch Lồi Nguyên Hỗn hợp thông qua Nới lỏng Phối cảnh

Việc giải quyết trực tiếp bài toán dưới dạng Quy hoạch Phi tuyến Nguyên Hỗn hợp (Mixed-Integer Nonlinear Programming - MINLP) đặt ra những thách thức tính toán đáng kể do sự hiện diện của các ràng buộc không lồi (non-convex constraints), đặc biệt là mối quan hệ song tuyến tính (bilinear relationship) giữa các biến quyết định nhị phân y_e và các biến trạng thái liên tục x_u . Để khắc phục điều này và chuyển đổi bài toán sang dạng Quy hoạch Lồi Nguyên Hỗn hợp (Mixed-Integer Convex Programming - MICP) có thể giải quyết được, chúng tôi áp dụng kỹ thuật nới lỏng phối cảnh

(perspective relaxation). Động lực chính của phương pháp này là cô lập tính không lồi bằng cách “nâng” (lifting) các biến liên tục vào một không gian có chiều cao hơn. Thay vì sử dụng các biến toàn cục x_u , chúng tôi giới thiệu các biến đại diện cục bộ (local proxy variables) $z_{e,u}, z_{e,v} \in \mathbb{R}^n$ cho mỗi cạnh $e = (u, v)$, đại diện cho sự đóng góp trạng thái của các đỉnh tương ứng khi cạnh đó được kích hoạt.

Đối với mỗi cạnh $e = (u, v)$, các biến đại diện cục bộ $z_{e,u}$ và $z_{e,v}$ mô hình hóa cụ thể sự đóng góp của trạng thái tại các đỉnh u và v vào việc di chuyển qua cạnh e . Nếu cạnh không được kích hoạt (tức là $y_e = 0$), các đóng góp này bị buộc về không. Mối quan hệ này được biểu diễn như sau:

$$z_{e,u} \in y_e X_u \quad \text{và} \quad z_{e,v} \in y_e X_v$$

điều này ngụ ý:

$$\begin{cases} z_{e,u} \in X_u & \text{nếu } y_e = 1 \\ z_{e,u} = 0 & \text{nếu } y_e = 0 \end{cases}$$

Logic này được hình thức hóa bằng cách sử dụng hàm phối cảnh (perspective function) $\tilde{\ell}_e((z_{e,u}, z_{e,v}), y_e)$, là phối cảnh của hàm chi phí gốc ℓ_e . Một tính chất cơ bản trong giải tích lồi (convex analysis) phát biểu rằng nếu ℓ_e là lồi, thì phối cảnh $\tilde{\ell}_e$ của nó là lồi đồng thời (jointly convex) theo tất cả các đối số. Tính chất này rất quan trọng vì nó cho phép sử dụng các bộ giải lồi (convex solvers) hiệu quả. Hàm phối cảnh được định nghĩa là:

$$\tilde{\ell}_e(z_{e,u}, z_{e,v}, y_e) = \begin{cases} y_e \ell_e\left(\frac{z_{e,u}}{y_e}, \frac{z_{e,v}}{y_e}\right) & \text{nếu } y_e > 0 \\ 0 & \text{nếu } y_e = 0 \text{ và } z_{e,u} = z_{e,v} = 0 \\ +\infty & \text{trường hợp khác} \end{cases}$$

Quan trọng hơn, thay vì áp đặt các ràng buộc đẳng thức tường minh như $z_{e,u} = y_e x_u$, vốn không lồi, chúng tôi mở rộng luật bảo toàn luồng (flow

conservation law) chuẩn (4.8b) sang các biến liên tục. Điều này bắt buộc rằng đối với bất kỳ nút u nào (ngoại trừ nguồn và đích), tổng các trạng thái liên tục đi vào nút thông qua các cạnh đến phải bằng tổng các trạng thái liên tục rời khỏi nút thông qua các cạnh đi.

Công thức MICP kết quả cho Bài toán Đường đi Ngắn nhất trong GCS được định nghĩa như sau:

$$\text{minimize} \quad \sum_{e=(u,v) \in \mathcal{E}} \tilde{\ell}_e((z_{e,u}, z_{e,v}), y_e) \quad (4.15a)$$

$$\text{subject to} \quad \sum_{e \in \mathcal{E}^+(u)} y_e - \sum_{e \in \mathcal{E}^-(u)} y_e = \delta_u, \quad \forall u \in \mathcal{V} \quad (4.15b)$$

$$\sum_{e \in \mathcal{E}^+(u)} z_{e,u} - \sum_{e \in \mathcal{E}^-(u)} z_{e,v} = \begin{cases} x_s & \text{nếu } u = s \\ -x_t & \text{nếu } u = t \\ 0 & \text{trường hợp khác} \end{cases}, \quad \forall u \in \mathcal{V} \quad (4.15c)$$

$$(z_{e,u}, z_{e,v}) \in y_e \mathcal{X}_e, \quad \forall e \in \mathcal{E} \quad (4.15d)$$

$$z_{e,u} \in y_e X_u, \quad z_{e,v} \in y_e X_v, \quad \forall e = (u, v) \in \mathcal{E} \quad (4.15e)$$

$$y_e \in \{0, 1\}, \quad \forall e \in \mathcal{E} \quad (4.15f)$$

Trong mô hình này, Phương trình (4.15b) đảm bảo tính liên thông tô pô (topological connectivity), trong đó δ_u được định nghĩa là 1 cho nguồn, -1 cho đích, và 0 cho các trường hợp khác. Phương trình (4.15c) đảm bảo rằng quỹ đạo liên tục là liên mạch trên cấu trúc đồ thị. Các phương trình (4.15d) và (4.15e) thực thi sự bao hàm hình học (geometric containment) bên trong các tập lỗi đã được tỉ lệ hóa; cụ thể, nếu $y_e = 0$, các biến liên tục liên quan bị ràng buộc về gốc tọa độ, phát sinh chi phí bằng không. Công thức này cung cấp một sự nói lỏng chặt chẽ và hiệu quả về mặt tính toán, cho phép bộ giải xác định các quỹ đạo tối ưu gần toàn cục (near-global optimal)

thỏa mãn cả tô pô đồ thị rời rạc và các ràng buộc liên tục phức tạp vốn có trong lập kế hoạch chuyển động.

4.3.6 Lỗi hóa Bài toán GCS thông qua Kỹ thuật RLT

Để giải quyết tính không lồi vốn có của Chương trình Phi lồi Nguyên Hỗn hợp (Mixed-Integer Non-Convex Program - MINCP) ban đầu—cụ thể là các ràng buộc song tuyến tính liên kết các biến quyết định nhị phân với các biến trạng thái liên tục—chúng tôi sử dụng chiến lược lỗi hóa (convexification strategy) dựa trên Kỹ thuật Cải biến-Tuyến tính hóa (Reformulation-Linearization Technique - RLT) cấp độ một. Mục tiêu cơ bản của phương pháp này là thay thế các ràng buộc khó giải bằng các bao lồi (convex envelopes) chặt chẽ. Điều này đảm bảo rằng Chương trình Lồi Nguyên Hỗn hợp (MICP) kết quả bảo toàn tập khả thi nguyên của bài toán gốc đồng thời cung cấp các cận dưới mạnh (strong lower bounds) khi các ràng buộc nguyên được nối lỏng.

4.3.6.1 Cơ sở Lý thuyết: Tính Đối ngẫu giữa Nón Phối cảnh và Bất đẳng thức Hợp lệ

Trước khi đi vào chi tiết mô hình tối ưu hóa, chúng ta phải thiết lập mối liên hệ toán học giữa các nón phối cảnh (perspective cones) và các bất đẳng thức tuyến tính. Xét nón đối ngẫu (dual cone) của nón phối cảnh $\tilde{X}$, ký hiệu là $(\tilde{X})^*$. Nón này được định nghĩa là tập hợp các vectơ (a, b) trong không gian mở rộng sao cho tích vô hướng của chúng với mọi phần tử trong nón $\tilde{X}$ là không âm:

$$(\tilde{X})^* := \{(a, b) \in \mathbb{R}^{n+1} \mid (a, b)^\top k \geq 0, \quad \forall k \in \tilde{X}\}. \quad (4.16)$$

Một kết quả quan trọng được sử dụng trong công thức này là sự tương đương giữa nón đối ngẫu $(\tilde{X})^*$ và tập hợp các bất đẳng thức tuyến tính

hợp lệ (valid linear inequalities) cho tập X . Về mặt hình học, điều này đại diện cho giao của tất cả các nửa không gian đóng (closed half-spaces) chứa X . Chúng tôi hình thức hóa điều này trong mệnh đề sau:

Mệnh đề. Nón đối ngẫu $(\tilde{X})^*$ chính xác là tập hợp các hệ số (a, b) xác định các bất đẳng thức tuyến tính hợp lệ trên tập X ; nghĩa là, $(\tilde{X})^* \equiv X^\circ$.

Chứng minh. Để chứng minh điều này, xét một phần tử tùy ý $k \in \tilde{X}$. Theo định nghĩa của nón phối cảnh, k có thể được biểu diễn dưới dạng $k = (\lambda x, \lambda)$ với một $x \in X$ nào đó và một vô hướng $\lambda \geq 0$. Điều kiện để một cặp vectơ (a, b) thuộc về nón đối ngẫu $(\tilde{X})^*$ có thể được khai triển như sau:

$$\begin{aligned} (a, b) \in (\tilde{X})^* &\iff (a, b)^\top (\lambda x, \lambda) \geq 0, \quad \forall x \in X, \forall \lambda \geq 0 \\ &\iff \lambda(a^\top x + b) \geq 0, \quad \forall x \in X, \forall \lambda \geq 0. \end{aligned} \quad (4.17)$$

Quan sát thấy rằng bất đẳng thức cuối cùng phải thỏa mãn với mọi giá trị không âm của λ . Trong trường hợp $\lambda > 0$, ta có thể chia cả hai vế cho λ mà không làm thay đổi dấu bất đẳng thức, thu được điều kiện $a^\top x + b \geq 0$. Trường hợp $\lambda = 0$ được thỏa mãn một cách hiển nhiên. Do đó, điều kiện cần và đủ để $(a, b) \in (\tilde{X})^*$ là biểu thức tuyến tính $a^\top x + b$ vẫn không âm trên toàn bộ tập X . Điều này xác nhận rằng $(\tilde{X})^*$ cấu thành tập hợp các hệ số cho tất cả các bất đẳng thức hợp lệ trên X , hoàn tất chứng minh.

4.3.6.2 Xây dựng các Ràng buộc Lỗi

Tận dụng mệnh đề trên, chúng ta có thể “nâng” bất kỳ ràng buộc tuyến tính hợp lệ nào từ không gian biến luồng (flow variable space) vào không gian hỗn hợp trạng thái-luồng (mixed state-flow space). Xét một đỉnh v tùy ý và một bất đẳng thức tuyến tính hợp lệ liên quan đến các biến luồng y_e kề với v , được cho bởi:

$$\sum_{e \in E_v} c_e y_e + d \geq 0, \quad (4.18)$$

trong đó E_v biểu thị tập hợp các cạnh kề với v , và d là một hằng số phụ thuộc vào bài toán. Bằng cách áp dụng nguyên lý phối cảnh, chúng tôi chuyển đổi bất đẳng thức (4.18) thành một ràng buộc bên trong nón lồi $\tilde{X}_v$. Để tạo thuận lợi cho việc này, chúng tôi giới thiệu các biến đại diện z_e và z'_e . Biến z_e đại diện cho sự đóng góp trạng thái tại đỉnh nguồn của cạnh e khi được kích hoạt, trong khi z'_e đại diện cho sự đóng góp trạng thái tại đỉnh đích.

Sự chuyển đổi này mang lại ràng buộc nón lồi sau:

$$\left(\sum_{e \in E_{\text{in}}(v)} c_e z'_e + \sum_{e \in E_{\text{out}}(v)} c_e z_e + dx_v, \sum_{e \in E_v} c_e y_e + d \right) \in \tilde{X}_v. \quad (4.19)$$

Phương trình (4.19) ràng buộc mối quan hệ giữa các biến trạng thái trung gian (z, z') và các biến luồng y , đảm bảo chúng nằm trong nón phối cảnh của tập khả thi X_v .

Một ứng dụng cụ thể nhưng quan trọng của Bổ đề này liên quan đến ràng buộc không âm của luồng, tức là $y_e \geq 0$. Đối với một cạnh $e = (u, v)$:

- Tại đỉnh nguồn u , áp dụng việc nâng với $c_e = 1$ mang lại ràng buộc $(z_e, y_e) \in \tilde{X}_u$.
- Tại đỉnh đích v , áp dụng việc nâng với $c_e = 1$ mang lại ràng buộc $(z'_e, y_e) \in \tilde{X}_v$.

Các ràng buộc này hoạt động hiệu quả như một cơ chế “bật/tắt”: nếu $y_e = 1$, biến đại diện z_e bị ràng buộc vào tập X_u ; ngược lại, nếu $y_e = 0$, z_e bị buộc về gốc tọa độ.

4.3.6.3 Công thức MICP Hoàn chỉnh

Tổng hợp các phân tích trước đó, chúng tôi đề xuất công thức Quy hoạch Lồi Nguyên Hỗn hợp (MICP) cho Bài toán Đường đi Ngắn nhất trên GCS, như được mô tả trong Định lý 5.7. Mô hình này loại bỏ hoàn toàn các

thành phần song tuyến tính, thay thế chúng bằng các ràng buộc nón lồi:

$$\underset{y, z, z'}{\text{minimize}} \quad \sum_{e \in E} \tilde{\ell}_e((z_e, z'_e), y_e) \quad (5.5a)$$

$$\text{subject to} \quad \sum_{e \in E_{\text{out}}(s)} y_e = 1, \quad \sum_{e \in E_{\text{in}}(t)} y_e = 1, \quad (5.5b)$$

$$\sum_{e \in E_{\text{out}}(v)} y_e \leq 1, \quad \forall v \in V \setminus \{s, t\}, \quad (5.5c)$$

$$\sum_{e \in E_{\text{in}}(v)} (z'_e, y_e) = \sum_{e \in E_{\text{out}}(v)} (z_e, y_e), \quad \forall v \in V \setminus \{s, t\}, \quad (5.5d)$$

$$(z_e, y_e) \in \tilde{X}_u, \quad (z'_e, y_e) \in \tilde{X}_v, \quad \forall e = (u, v) \in E, \quad (5.5e)$$

$$y_e \in \{0, 1\}, \quad \forall e \in E. \quad (5.5f)$$

Công thức (4.20) bao gồm bốn thành phần chính đảm bảo tính liên thông đồ thị và tính nhất quán hình học.

Thứ nhất, hàm mục tiêu (5.5a) đại diện cho tổng chi phí trên tất cả các cạnh, trong đó $\tilde{\ell}_e$ là hàm phối cảnh của thước đo chi phí gốc. Vì các phép toán phối cảnh bảo toàn tính lồi, bài toán tối ưu hóa duy trì được cấu trúc lồi có thể giải quyết được.

Thứ hai, các ràng buộc (5.5b) và (5.5c) thực thi bảo toàn luồng cổ điển, đảm bảo sự tồn tại của một đường đi từ nguồn s đến đích t trong khi hạn chế mỗi đỉnh trung gian chỉ được ghé thăm tối đa một lần.

Thứ ba, ràng buộc (5.5d) đại diện cho bảo toàn luồng không gian (spatial flow conservation), một đặc điểm khác biệt so với các bài toán đường đi ngắn nhất tiêu chuẩn. Ràng buộc này được suy ra bằng cách nhân phương trình bảo toàn luồng tại đỉnh v với biến trạng thái x_v . Về mặt vật lý, nó đảm bảo tính liên tục của quỹ đạo: trạng thái tổng hợp “đi vào” đỉnh v (thông qua z'_e) phải bằng trạng thái tổng hợp “rời khỏi” đỉnh v (thông qua z_e).

Cuối cùng, các ràng buộc (5.5e) là kết quả của quá trình lồi hóa đã thảo

luận trước đó. Chúng đóng vai trò là các bao lồi cho các biến đại diện z_e và z'_e , đảm bảo rằng khi một cạnh $e = (u, v)$ được chọn ($y_e = 1$), các biến này tương ứng với các điểm hợp lệ nằm trong các tập X_u và X_v .

4.4 Thiết kế thực nghiệm

Mục đích của các thực nghiệm này là kiểm chứng hiệu quả của phương pháp GCS trong bài toán lập kế hoạch quỹ đạo có ràng buộc động lực học. Chúng tôi tập trung đánh giá hai yếu tố cốt lõi: chất lượng của quá trình phân hoạch không gian tự do ($\mathcal{X}_{free}$) cũng như tác động của nó lên mức độ tối ưu của quỹ đạo chuyển động đầu ra trên môi trường 2D đơn giản và khả năng tối ưu hóa quỹ đạo toàn cục trong môi trường có mức độ phức tạp tổ hợp lớn.

4.4.1 Môi trường 2D đơn giản

Chúng tôi xem xét một không gian làm việc hai chiều đơn giản với cấu hình bắt đầu q_0 và cấu hình kết thúc q_T được xác định trước. Không gian tự do $\mathcal{X}_{free}$ được phân rã thành tập hợp các vùng lồi an toàn $\{Q_i\}_{i \in I}$ thông qua các thuật toán phân hoạch khác nhau để làm cơ sở cho việc thiết lập đồ thị.

Trong kịch bản này, chúng tôi tập trung phân tích giai đoạn tiền xử lý không gian cấu hình. Chất lượng của các vùng lồi được tạo ra là yếu tố quyết định số lượng biến nguyên trong bài toán MICP, từ đó ảnh hưởng trực tiếp đến thời gian giải và tính tối ưu của quỹ đạo.

Mục tiêu: Định lượng sự tác động của các thuật toán phân hoạch lồi khác nhau (ACD, VCC) so với phương pháp phân hoạch thủ công³ (Manual) lên hiệu suất của bộ lập kế hoạch GCS.

³Tức người dùng sẽ tự phân hoạch không gian tự do. Ở đây chúng tôi giả sử kết quả từ phân hoạch thủ công là kết quả tốt nhất để từ đó so sánh mức độ hiệu quả của các giải thuật còn lại.

Mô tả kịch bản: Thiết lập một không gian làm việc 2D với các vật cản đa giác lồi được bố trí cố định. Cùng với đó thực hiện phân hoạch không gian tự do bằng ba phương pháp:

1. **Phân hoạch Thủ công:** Tạo ra số lượng vùng lồi tối thiểu bằng sự can thiệp của con người để làm chuẩn đối sánh.
2. **Phân hoạch Lồi Xấp xỉ (ACD):** Sử dụng thuật toán với mức dung sai $\tau = 0.0$ (tức các vùng lồi được tạo ra hoàn toàn lồi).
3. **Lớp phủ Clique Khả kiến (VCC):** Sử dụng VCC với mức độ phủ của tập các vùng lồi là 96%.

Thiết lập bài toán tối ưu: Chúng tôi tập trung vào bài toán tối ưu hóa thời gian thực hiện (minimum-time problem) kết hợp với các ràng buộc động lực học chặt chẽ. Các tham số cấu hình hệ thống được thiết lập như sau:

1. **Hàm mục tiêu và chi phí:** Sử dụng hàm chi phí thời gian $J_t = w_t T_e$ với trọng số $w_t = 1$.
2. **Tham số hóa quỹ đạo:** Quỹ đạo được mô hình hóa bằng các đường cong Bézier bậc $d = 6$.
3. **Ràng buộc liên tục:** Áp đặt độ liên tục cấp C^2 tại các điểm chuyển tiếp giữa các tập lồi.
4. **Giới hạn động lực học:** Vận tốc trên mỗi chiều bị giới hạn trong khoảng $\dot{q} \in [-1, 1]$.
5. **Điều chuẩn và độ trơn:** Áp dụng điều chuẩn đạo hàm bậc hai với trọng số $\epsilon = 10^{-1}$.

Bằng cách giải bài toán nói lỏng lồi của SPP trong khung làm việc GCS, chúng tôi thu được quỹ đạo tối ưu không va chạm, đồng thời cung cấp cận

sai số tối ưu (optimality gap) để chứng minh tính hiệu quả của phương pháp so với nghiệm tối ưu toàn cục.

Chỉ số đo lường:

1. **Hiệu quả phân hoạch:** Số lượng vùng lỗi (k) và thời gian tính toán của thuật toán phân hoạch.
2. **Hiệu suất tối ưu hóa:** Thời gian giải bài toán MICP/SOCP.
3. **Chất lượng quỹ đạo:** Giá trị chi phí thời gian và giá trị tối ưu của hàm mục tiêu.

4.4.2 Môi trường mê cung

Trong thực nghiệm này, chúng tôi mở rộng phạm vi kiểm chứng sang một môi trường có độ phức tạp tổ hợp cao hơn đáng kể: một mê cung lớn với cấu trúc hình học dày đặc.

Mục tiêu: Minh họa khả năng của framework GCS trong môi trường có độ phức tạp tổ hợp lớn và khả năng kết hợp tối ưu hóa rời rạc với tối ưu hóa liên tục.

Mô tả kịch bản: Cấu trúc môi trường: Mê cung kích thước $25 \times 25 = 625$ ô, sinh bằng thuật toán DFS ngẫu nhiên và loại bỏ ngẫu nhiên 100 bức tường. Mỗi ô được mô hình hóa thành một tập an toàn lỗi Q_i với các cạnh hai chiều giữa các ô kề nhau.

Thiết lập bài toán tối ưu:

1. **Bậc đa thức và độ trơn:** Đường cong Bézier bậc 6 với ràng buộc liên tục cấp C^2 .
2. **Hàm mục tiêu:** Tối thiểu hóa thời gian kết hợp điều chuẩn đạo hàm bậc hai với $\epsilon = 10^{-1}$.
3. **Ràng buộc vi phân:** Vận tốc giới hạn trong $[-1, 1]$, với $\dot{h}_{min} = 10^{-1}$.
4. **Điều kiện biên:** Vận tốc tại điểm đầu và cuối bằng 0.

5. **Bộ giải:** Sử dụng MosekSolver để giải bài toán SOCP.

Chỉ số đo lường:

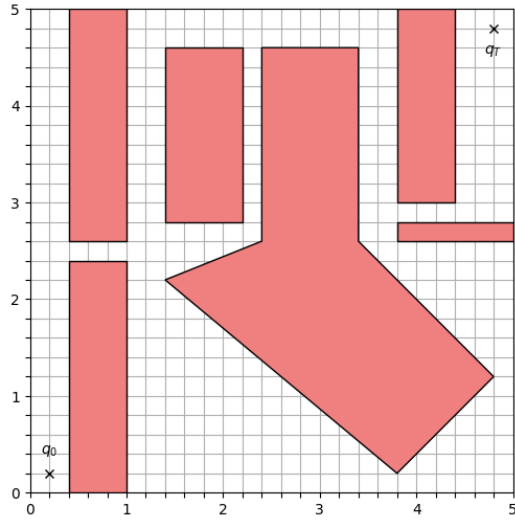
1. Thời gian giải (t_{solve}).
2. Cận sai số tối ưu ($\delta_{relax} = \frac{C_{round} - C_{relax}}{C_{relax}}$).
3. Tổng chi phí thời gian (T).
4. Độ dài quỹ đạo hình học (L).

4.5 Kết quả và Phân tích

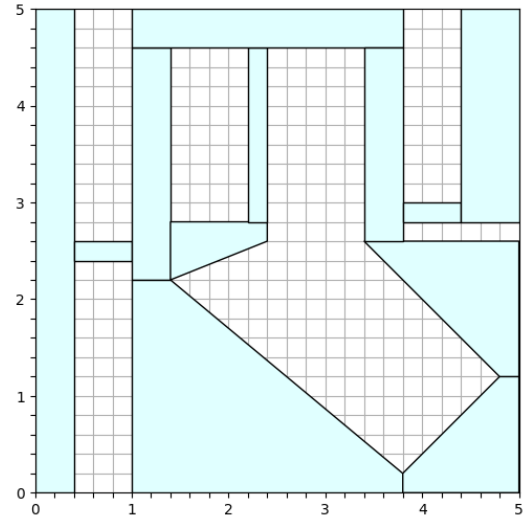
Trong chương này, chúng tôi trình bày các kết quả định lượng thu được từ các thực nghiệm đã thiết lập ở phần trước. Mục tiêu là phân tích sự đánh đổi giữa độ phức tạp của việc phân hoạch không gian và tính tối ưu của quỹ đạo đầu ra. Toàn bộ các thực nghiệm được thực hiện trên máy tính cá nhân với cấu hình: CPU AMD Ryzen 7 4800H 2.9 GHz, RAM 16GB, hệ điều hành Ubuntu 24.04.

4.5.1 Phân tích Môi trường 2D đơn giản: Tác động của Phân hoạch lỗi

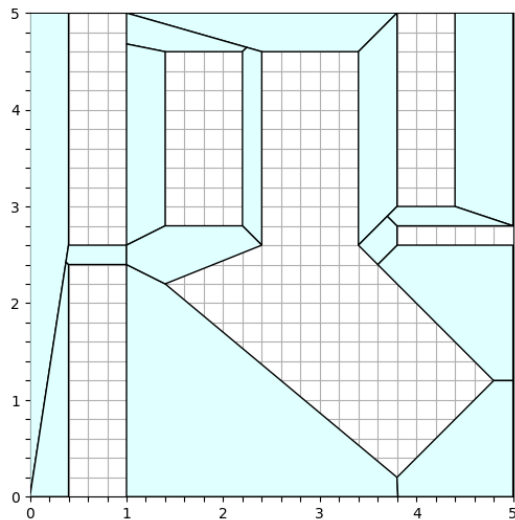
Kết quả thực nghiệm đối với các giải thuật phân hoạch tự động cho thấy sự tương phản rõ rệt về chiến lược biểu diễn không gian cấu hình cũng như chi phí tính toán giữa hai phương pháp tiếp cận chủ đạo là ACD và VCC. Đối với giải thuật Phân hoạch Lỗi Xấp xỉ (ACD), kết quả đạt được thể hiện như hình 4.10c ưu thế vượt trội trong môi trường hình học 2D khi chỉ tạo ra 15 vùng lỗi, một con số tiệm cận sát với mức tối ưu của phương pháp phân hoạch thủ công là 12 vùng. Hiệu suất này được củng cố bởi thời gian xử lý nhanh, chỉ xấp xỉ 0,01 giây, hoàn toàn phù hợp với độ phức tạp lý thuyết $O(nr^2)$ [8] của thuật toán khi xử lý các đa giác có số lượng đỉnh



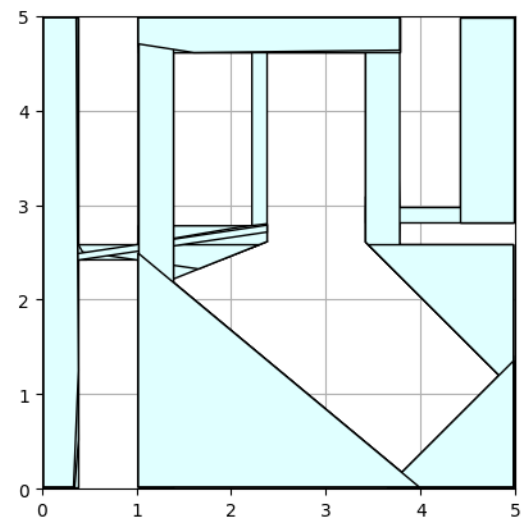
(a) Không gian 2D với vật cản (vùng màu đỏ) ban đầu



(b) Kết quả Phân hoạch thủ công



(c) Kết quả Phân hoạch ACD



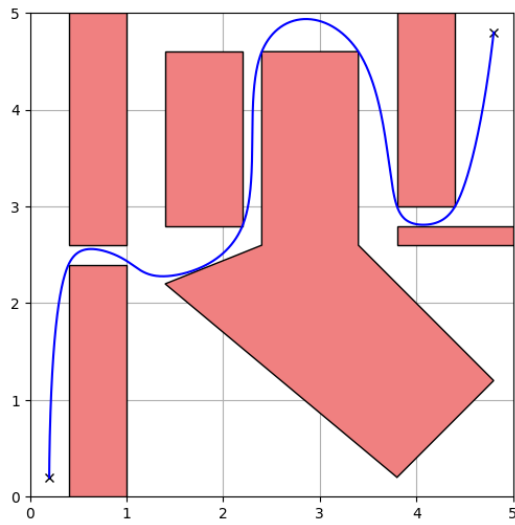
(d) Kết quả Phân hoạch VCC

Hình 4.10: So sánh chất lượng các vùng lõi được phân hoạch bởi các giải thuật khác nhau. Số lượng tập lõi tăng dần từ (b) đến (d), cho thấy sự cải thiện về độ chính xác nhưng tăng độ phức tạp tính toán.

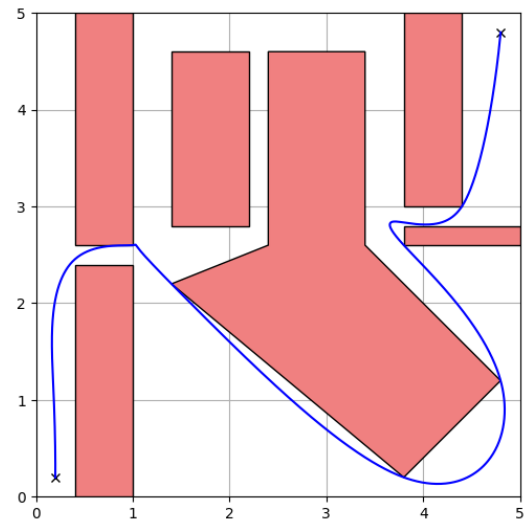
lỗm không quá lớn. Việc ACD có khả năng kiểm soát mức độ chi tiết thông qua tham số lỗm τ cho phép hệ thống duy trì được sự cân bằng giữa độ chính xác hình học và số lượng vùng lỗi tối giản, từ đó giảm thiểu số lượng biến nhị phân trong bài toán quy hoạch nguyên-hỗn hợp (MICP) của GCS framework.

Ngược lại, giải thuật Visibility VCC (VCC) lại mang đến một cấu trúc phân hoạch dày đặc và phức tạp hơn với số lượng vùng lỗi dao động từ 33 đến 35 vùng. Sự thiếu ổn định về số lượng vùng phân hoạch giữa các lần thực thi xuất phát trực tiếp từ tính chất ngẫu nhiên của bước lấy mẫu n điểm trong không gian cấu hình tự do C_{free} để xây dựng đồ thị khả kiến đã được nêu ở phần 4.2.3. Mặc dù sự chênh lệch khoảng 1-3 vùng lỗi tại các khu vực hành lang hẹp là mức sai số có thể chấp nhận được, nhưng tổng số lượng vùng lỗi lớn đã tạo ra một gánh nặng tính toán đáng kể cho bộ giải Mosek do số lượng biến nhị phân y_e tăng lên theo tỉ lệ xấp xỉ $2|I|$. Thời gian hội tụ của VCC trong môi trường 2D thực tế không mang lại hiệu quả cao cho các ứng dụng yêu cầu phản hồi tức thời so với các kỹ thuật phân hoạch đa giác trực tiếp như ACD. Tuy nhiên, cần phải nhận rằng VCC được thiết kế như một phương pháp lai ghép mạnh mẽ để xử lý các không gian cấu hình cao chiều, nơi các vật cản không được xác định tường minh bằng các đa giác mà thông qua các bộ kiểm tra va chạm chung. Trong các hệ thống có số chiều lớn như cánh tay robot 7 bậc tự do (7-DoF), việc lấy mẫu và tìm lớp phủ clique giúp VCC tự động hóa quá trình đặt các điểm mào chiến lược cho thuật toán IRIS, điều mà các phương pháp phân hoạch hình học 2D đơn giản không thể thực hiện được. Do đó, việc áp dụng VCC vào môi trường 2D trong thực nghiệm này chủ yếu đóng vai trò kiểm chứng khả năng bao phủ toàn cục và tính an toàn của quỹ đạo.

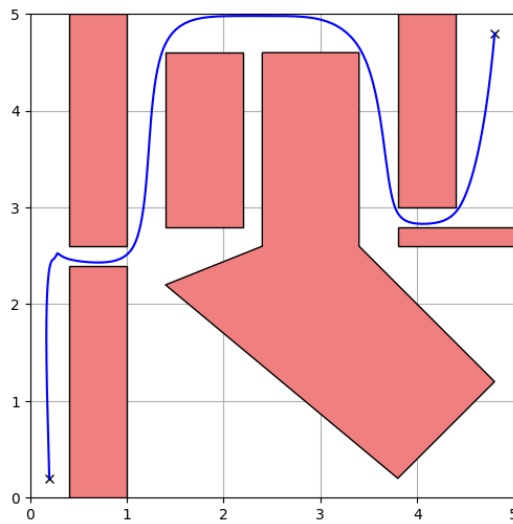
Sau khi hoàn tất giai đoạn phân hoạch không gian tự do thành các tập hợp vùng lỗi, chúng tôi tiến hành thực hiện bài toán hoạch định quỹ đạo dựa



(a) Quỹ đạo chuyển động với phân hoạch thủ công



(b) Quỹ đạo chuyển động với phân hoạch ACD



(c) Quỹ đạo chuyển động với phân hoạch VCC

Hình 4.11: So sánh chất lượng các quỹ đạo được hoạch định cho tập các vùng lồi được tạo bởi các giải thuật ở trên.

Bảng 4.1: So sánh hiệu năng giữa các phương pháp lập kế hoạch quỹ đạo

Phương pháp	Thời gian phân hoạch (s)	Số vùng	Thời gian giải (s)	Time cost (s)	Path cost (rounded)	Tổng thời gian (s)
VCC	5.7807	33	269.9363	13.2217	24.4365	275.7170
ACD	0.0109	15	4.3105	12.7875	24.6286	4.3214
Phân hoạch thủ công	—	12	3.9005	13.3565	27.8805	3.9005

trên khung làm việc Đồ thị các Tập lồi (Graphs of Convex Sets - GCS). Mục tiêu của giai đoạn này là tìm kiếm một lộ trình tối ưu không chỉ về mặt hình học mà còn phải đáp ứng các ràng buộc về động lực học và độ trơn của quỹ đạo. Để đạt được sự cân bằng giữa thời gian thực thi vật lý ($T_{duration}$) và nỗ lực điều khiển (Control effort), hàm mục tiêu tổng hợp (Rounded Cost) được thiết lập như sau:

$$RoundedCost \approx (1 \times T_{duration}) + \sum \left(0.1 \times \int_0^{T_e} \|a(t)\|^2 dt \right)$$

Trong đó, trọng số cho thời gian thực thi được đặt là $w_t = 1$ và hệ số điều chuẩn đạo hàm bậc hai (gia tốc) là $\epsilon = 0.1$ nhằm giảm thiểu hiện tượng rung động (jerk) khi robot di chuyển qua các ranh giới vùng lồi. Các kết quả định lượng về hiệu suất tính toán và chất lượng quỹ đạo được phân tích chi tiết dựa trên dữ liệu thu được từ bộ giải Mosek.

4.5.1.1 Hiệu suất tính toán và Khả năng đáp ứng thời gian thực

Kết quả thực nghiệm cho thấy một sự tương quan phi tuyến tính chặt chẽ giữa số lượng vùng lồi được phân hoạch và thời gian hội tụ của bộ giải. Phương pháp **VCC** cho thấy hiệu năng kém nhất trong môi trường thực nghiệm 2D. Mặc dù số lượng vùng lồi được tạo ra (33 vùng) chỉ cao hơn gấp đôi so với ACD, nhưng thời gian giải bài toán tối ưu hóa lại rất lớn lên tới xấp xỉ 269,94 giây. Hiện tượng này xuất phát từ việc gia tăng số lượng vùng lồi dẫn đến sự bùng nổ của các biến quyết định nhị phân và các ràng buộc nón phối cảnh liên kết, khiến không gian tìm kiếm tổ hợp vượt quá khả năng xử lý hiệu quả của bộ giải trong thời gian thực.

Ngược lại, phương pháp **ACD** thể hiện tính thực tiễn vượt trội khi tổng thời gian thực thi (bao gồm cả phân hoạch và giải quỹ đạo) chỉ mất 4,32 giây, nhanh gấp khoảng 64 lần so với VCC. Kết quả này khẳng định rằng đối với các ứng dụng di động trong môi trường 2D, một cấu trúc phân hoạch tối giản (15 vùng) là yếu tố tiên quyết để duy trì khả năng phản hồi

nhánh của hệ thống. Trong khi đó, phương pháp **Phân hoạch thủ công** mặc dù đạt thời gian giải nhanh nhất (3,90 giây) do cấu trúc đồ thị đơn giản (12 vùng), nhưng không được xem là giải pháp khả thi cho các hệ thống tự hành do đòi hỏi sự can thiệp trực tiếp từ con người, vốn là một rào cản lớn đối với tính tự chủ của robot.

4.5.1.2 Chất lượng Quỹ đạo và Sự đánh đổi giữa các Mục tiêu Tối ưu

Chất lượng của quỹ đạo được đánh giá thông qua sự đánh đổi giữa thời gian di chuyển thực tế và giá trị tối ưu của hàm mục tiêu (Path cost).

- **Phương pháp VCC:** Đạt giá trị Path cost thấp nhất (24,44), đại diện cho nghiệm tối ưu nhất về mặt toán học. Việc chia nhỏ không gian thành nhiều vùng lõi diện tích nhỏ cho phép bộ giải có nhiều "không gian tự do" hơn để tính chỉnh đường cong Bézier, từ đó tối ưu hóa nỗ lực điều khiển và giảm thiểu tối đa các pha phạt gia tốc. Tuy nhiên, thời gian thực thi vật lý (13,22 giây) lại chậm hơn ACD, cho thấy hệ thống chấp nhận di chuyển với vận tốc thấp hơn để đổi lấy độ trơn tuyệt đối.
- **Phương pháp ACD:** Mang lại kết quả thực dụng nhất khi robot hoàn thành nhiệm vụ với thời gian ngắn nhất (12,79 giây). Mặc dù Path cost (24,63) cao hơn nhẹ so với VCC, nhưng sự chênh lệch này không đáng kể trong vận hành thực tế. Điều này chứng tỏ ACD đã tạo ra một quỹ đạo "thực dụng": chạy nhanh, đủ mượt mà và duy trì được tính toán hiệu quả.
- **Phân hoạch thủ công:** Thể hiện hiệu năng kém nhất về mặt chất lượng quỹ đạo với Path cost cao nhất (27,88) và thời gian di chuyển lâu nhất (13,36 giây). Nguyên nhân chủ yếu nằm ở việc phân chia vùng dựa trên trực giác con người thường tạo ra các vùng lõi thô,

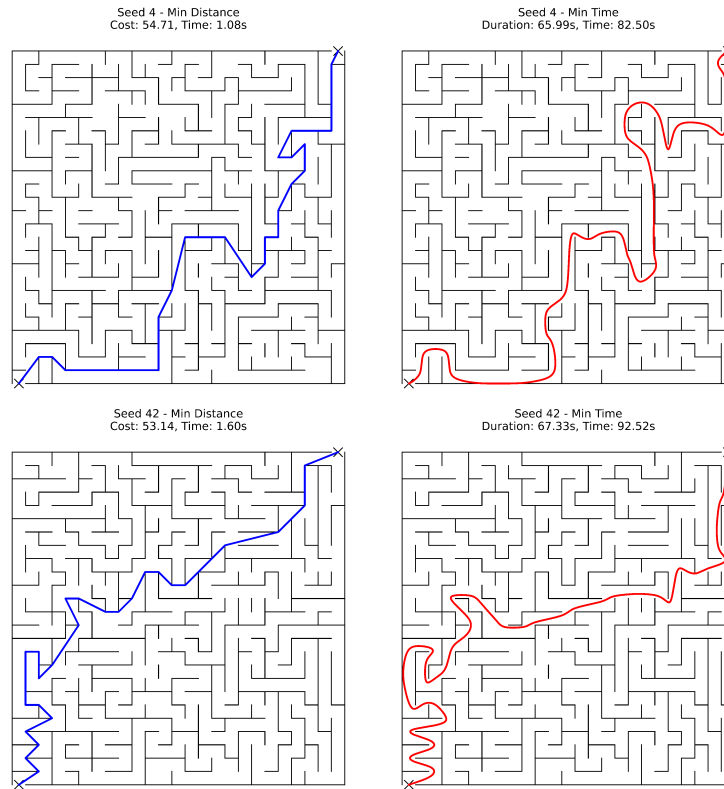
các vùng có thể to nhưng hình dạng không tối ưu cho robot lách qua, hoặc các điểm nối giữa các vùng bị đặt ở vị trí xấu, khiến robot phải đi đường vòng hoặc phanh gấp tại các giao điểm.

Tóm lại, thông qua các chỉ số đo lường thực nghiệm, chúng tôi xác định **ACD** là giải pháp tối ưu cho các bài toán hoạch định quỹ đạo 2D, đảm bảo được sự cân bằng giữa chi phí tính toán và hiệu suất vận hành thực tế. Trong khi đó, phương pháp VCC dù cho thấy tiềm năng về tính tối ưu toán học nhưng lại gặp rào cản lớn về thời gian giải, khiến nó phù hợp hơn cho các ứng dụng ngoại tuyến hoặc trong các không gian cấu hình cao chiều phức tạp.

4.5.2 Phân tích Thực nghiệm trên Môi trường Mê cung Phức tạp

Trong phần thực nghiệm này, chúng tôi mở rộng quy mô kiểm chứng sang môi trường mê cung có độ phức tạp tổ hợp lớn nhằm đánh giá khả năng của khung làm việc GCS trong việc kết hợp hài hòa giữa tối ưu hóa rời rạc và liên tục. Để đảm bảo tính khách quan và độ tin cậy về mặt thống kê, các thực nghiệm được thực hiện trên 10 cấu trúc mê cung khác nhau, được sinh ngẫu nhiên bằng thuật toán DFS với kích thước $25 \times 25 = 625$ ô. Trong mỗi mê cung, chúng tôi loại bỏ ngẫu nhiên 100 bức tường để tạo ra các lộ trình đa dạng, buộc bộ lập kế hoạch phải tìm kiếm nghiệm tối ưu trên một đồ thị có cấu trúc liên thông phức tạp.

Các tham số tối ưu hóa được thiết lập đồng nhất cho tất cả các mẫu thử nghiệm với đường cong Bézier bậc $d = 6$, đảm bảo độ trơn liên tục cấp C^2 . Hàm mục tiêu tập trung tối thiểu hóa thời gian thực thi kết hợp với điều chuẩn đạo hàm bậc hai ($\epsilon = 10^{-1}$) nhằm phạt các gia tốc quá lớn. Bộ giải *Mosek* được sử dụng để xử lý bài toán SOCP thu được từ kỹ thuật nói lỏng lồi.



Hình 4.12: Quỹ đạo chuyển động cho 1 số mê cung

4.5.2.1 Phân tích Hiệu suất Tính toán và Chi phí Tối ưu

Dựa trên dữ liệu thu thập từ 10 mẫu thử nghiệm, chúng tôi tiến hành so sánh đối chiếu giữa hai phương pháp:

- **Linear GCS:** tối thiểu hóa độ dài hình học.
- **Bezier GCS:** tối thiểu hóa thời gian với ràng buộc động lực học.

Bảng 4.2: Thống kê kết quả thực nghiệm trên 10 mẫu mê cung ngẫu nhiên

Chỉ số (Metric)	Linear GCS	Bezier GCS	Đơn vị
Thời gian giải trung bình (Mean)	1,28	85,22	s
Thời gian giải trung vị (Median)	1,16	85,10	s
Chi phí trung bình (Mean Cost)	71,88	88,03	unit
Sai số chi phí (Cost Ratio TB)	0,82	1,00	x

Kết quả thống kê cho thấy sự khác biệt rõ rệt về hiệu suất:

Về thời gian tính toán: Linear GCS cho thấy khả năng phản hồi cực nhanh với thời gian giải trung bình chỉ 1,28 giây. Trong khi đó, Bezier GCS yêu cầu trung bình 85,22 giây để hội tụ. Sự gia tăng này là do Bezier GCS

phải giải quyết các ràng buộc vi phân bậc cao (C^2) và tối ưu hóa đồng thời biến thời gian h , khiến bài toán SOCP có quy mô lớn hơn đáng kể.

Về chi phí tối ưu: Chi phí trung bình của Bezier GCS cao hơn khoảng 22% so với Linear GCS. Tuy nhiên, chi phí của Bezier GCS bao gồm cả thành phần điều chuẩn gia tốc (*Regularization Cost*), phản ánh sự đánh đổi cần thiết để đạt được quỹ đạo mượt mà và khả thi về mặt vật lý.

4.5.2.2 Phân tích Đặc điểm Quỹ đạo và Tính Tối ưu Toàn cục

Quan sát các quỹ đạo tại Hình 4.12 (tương ứng với các *Seeds* khác nhau), chúng tôi rút ra các nhận định quan trọng về hành vi của bộ lập kế hoạch:

1. **Sự khác biệt về lộ trình (Homotopy class):** Trong nhiều trường hợp (ví dụ *Seed 4* và *Seed 42*), quỹ đạo Bezier GCS (màu đỏ) lựa chọn một lộ trình hoàn toàn khác so với quỹ đạo Linear GCS (màu xanh). Trong khi quỹ đạo hình học ưu tiên đường ngắn nhất với nhiều khúc cua gắt, quỹ đạo Bezier chấp nhận đi quãng đường dài hơn nhưng ít góc rẽ gắt hơn nhằm duy trì vận tốc cao và giảm chi phí gia tốc.
2. **Hành vi tại các góc cua:** Quỹ đạo Bezier thể hiện xu hướng “ôm cua” rộng tại các giao lộ để đảm bảo tính liên tục của gia tốc (C^2). Các thành phần vận tốc được duy trì trong giới hạn $[-1, 1]$ nhờ vào cấu trúc nón phối cảnh trong mô hình GCS.
3. **Tính tối ưu toàn cục:** Đối với cả 10 mẫu mê cung, kỹ thuật nói lỏng lời đều trả về các xác suất nhị phân ϕ_e hội tụ về $\{0, 1\}$, giúp nghiệm thu được tự động đạt tối ưu toàn cục mà không cần qua bước làm tròn (*Rounding*). Điều này chứng minh GCS vượt trội hơn các phương pháp MICP truyền thống khi giảm số lượng biến nhị phân từ mức $|I|^2$ xuống chỉ còn $2|I|$, cho phép giải quyết các mê cung lớn mà các bộ giải cũ thường thất bại.

Kết luận, việc thực hiện trên 10 mê cung khác nhau đã khẳng định tính ổn

định của framework GCS. Mặc dù chi phí tính toán cho quỹ đạo *kinodynamic* cao hơn, GCS cung cấp một giải pháp nhất quán để tìm ra lộ trình trơn tru, an toàn tuyệt đối và tối ưu toàn cục trong các môi trường có độ phức tạp tổ hợp cực cao.

Chương 5

Conclusion

5.1 Summary

5.2 Future Work

Future research will focus on...

- ...
- ...

Tài liệu tham khảo

- [1] R. Deits and R. Tedrake, “Motion planning around obstacles with convex optimization,” *arXiv preprint*, vol. arXiv:2205.04422, 2022. [Online]. Available: <https://arxiv.org/abs/2205.04422>.
- [2] Boston Dynamics, *Leaps, bounds, and backflips*, Blog post, 2021. [Online]. Available: <https://bostondynamics.com/blog/leaps-bounds-and-backflips/>.
- [3] Amazon Robotics, *Amazon.com announces first quarter results*, Press release, 2023. [Online]. Available: <https://ir.aboutamazon.com/news-release/news-release-details/2023/Amazon.com-Announces-First-Quarter-Results/default.aspx>.
- [4] M. Diehl, H. G. Bock, H. Diedam, and P.-B. Wieber, “Fast direct multiple shooting algorithms for optimal robot control,” in *Fast Motions in Biomechanics and Robotics*, Springer, 2006, pp. 65–93. DOI: [10.1007/978-3-540-36119-0_4](https://doi.org/10.1007/978-3-540-36119-0_4).
- [5] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Overmars, “Probabilistic roadmaps for path planning in high-dimensional configuration spaces,” *IEEE Transactions on Robotics and Automation*, vol. 12, no. 4, pp. 566–580, 1996. DOI: [10.1109/70.508439](https://doi.org/10.1109/70.508439).
- [6] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge University Press, 2004. [Online]. Available: <https://web.stanford.edu/~boyd/cvxbook/>.

- [7] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, 1993. [Online]. Available: https://web.mit.edu/dimitrib/www/NETWORK_FLOW.pdf.
- [8] J. M. Lien and N. M. Amato, “Approximate convex decomposition of polygons,” *Computational Geometry: Theory and Applications*, 2006. [Online]. Available: https://cs.gmu.edu/~jmlien/masc/uploads/Main/cd2d_CGTA.pdf.
- [9] R. Deits and R. Tedrake, “Approximating robot configuration spaces with few convex sets using clique covers of visibility graphs,” *arXiv preprint*, vol. arXiv:2310.02875, 2023. [Online]. Available: <https://arxiv.org/pdf/2310.02875>.
- [10] R. Deits and R. Tedrake, “Computing large convex regions of obstacle-free space through semidefinite programming,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2014. [Online]. Available: https://groups.csail.mit.edu/robotics-center/public_papers/Deits14.pdf.
- [11] R. Deits and R. Tedrake, “Growing convex collision-free regions in configuration space using nonlinear programming,” *arXiv preprint*, vol. arXiv:2303.14737, 2023. [Online]. Available: <https://arxiv.org/abs/2303.14737>.
- [12] R. Deits and R. Tedrake, “Finding and optimizing certified, collision-free regions in configuration space for robot manipulators,” *arXiv preprint*, vol. arXiv:2205.03690, 2022. [Online]. Available: <https://arxiv.org/abs/2205.03690>.
- [13] T. Marcucci, J. Umenberger, P. A. Parrilo, and R. Tedrake, “Shortest paths in graphs of convex sets,” *arXiv preprint*, vol. arXiv:2101.11565, 2023. [Online]. Available: <https://arxiv.org/pdf/2101.11565>.

- [14] T. Marcucci, “Graphs of convex sets with applications to optimal control and motion planning,” Ph.D. dissertation, Massachusetts Institute of Technology, 2024. [Online]. Available: <https://dspace.mit.edu/handle/1721.1/156046>.

Chương A

...

A.1 ...

A.2 ...